.UBAfiuba
FACULTAD DE INGENIERÍA

# Integración de Técnicas de Visión Computacional y Aprendizaje Automático en un Robot Militar para la Identificación y Eliminación Eficiente de Objetivos

Lucas Martín Gaggino
Tomas Jesús Contessi

**Trabajo Profesional**
**Carrera de Ingeniería Electrónica**

**Tutor:** Federico G. Zacchigna

*Ciudad de Buenos Aires, Agosto de 2025*

# *Resumen*

Este trabajo profesional presenta el desarrollo de un sistema robótico militar autónomo capaz de identificar y neutralizar objetivos aéreos de manera eficiente mediante el uso de inteligencia artificial y visión por computadora. El sistema integra capacidades de navegación autónoma, reconocimiento de objetivos en tiempo real y control automático de armamento, coordinados por una arquitectura de software modular que permite la operación independiente y escalable de cada componente. La principal innovación del proyecto radica en la combinación de técnicas de aprendizaje automático con un entorno de simulación avanzado que permite entrenar y validar los algoritmos de manera segura y económica, superando las limitaciones tradicionales de desarrollo en entornos militares reales.

Los resultados demuestran que el sistema desarrollado puede operar de manera autónoma en diversos escenarios, navegando eficientemente mientras detecta y neutraliza objetivos con alta precisión. La importancia de este trabajo reside en su contribución al desarrollo de sistemas de defensa inteligentes que pueden reducir significativamente los riesgos para el personal militar y mejorar la efectividad operacional. El lector encontrará en esta memoria una descripción completa del proceso de desarrollo, desde la conceptualización teórica hasta la implementación práctica y validación experimental, incluyendo tanto el diseño del hardware como el desarrollo del software de control. El proyecto integra conocimientos fundamentales de la ingeniería electrónica, incluyendo sistemas de control, procesamiento de señales, programación de sistemas embebidos y diseño de circuitos, demostrando la aplicación práctica de estos conceptos en una solución tecnológica innovadora para aplicaciones de defensa.

# Índice general

**Resumen** I

**1. Presentación** 1
1.1. Introducción . . . 1
1.2. Antecedentes . . . 1
1.3. Estado del Arte . . . 2
1.4. Áreas Profesionales de Relevancia . . . 3
1.5. Alcance y objetivos . . . 3
1.5.1. Objetivos generales . . . 3
1.5.2. Objetivos particulares . . . 4

**2. Introducción Teórica** 5
2.1. Detección de imágenes . . . 5
2.2. Mapeo . . . 7
2.3. Path planning . . . 8
2.4. Seguimiento de ruta . . . 9
2.5. Simulación . . . 10

**3. Analisis técnico** 13
3.1. sección 1 . . . 13
3.2. sección 2 . . . 13

**4. Analisis de mercado** 15
4.1. Cliente objetivo primario en Argentina . . . 15
4.2. Mercados potenciales para el producto . . . 15
4.2.1. Sector público argentino . . . 15
4.3. Análisis de costos del prototipo . . . 16

**5. Prototipo** 19
5.1. Selección de componentes . . . 19
5.1.1. Ruedas . . . 19
5.1.2. motores de tracción . . . 20
5.1.3. Motores de la torreta . . . 21
5.1.4. Drivers de motor . . . 21
5.1.5. Interfaz con la PC . . . 21
5.1.6. Baterias . . . 21
5.2. Chasis . . . 22
5.2.1. Análisis de Implementación . . . 22
Ejes . . . 22
Largueros del chasis . . . 23
Pisos . . . 24
Coraza superior . . . 25
Torreta . . . 25

Ensamblaje completo . . . . . 26
5.2.2. Conclusiones del Análisis de Implementación . . . . . 27
5.3. Software . . . . . 27
5.3.1. Máquina de Estados Finitos . . . . . 28
Estados Principales . . . . . 28
5.3.2. Análisis de Implementación . . . . . 29
Control Central y Coordinación de Módulos . . . . . 29
Seguimiento de Objetivos y Control de Torreta . . . . . 30
Navegación Autónoma y Planificación de Rutas . . . . . 31
Comunicación y Mensajería entre Módulos . . . . . 32
Configuración y Modularidad del Sistema . . . . . 32
5.3.3. Conclusiones del Análisis de Implementación . . . . . 32
5.4. Interfaz y control de motores . . . . . 33
5.4.1. Análisis de Implementación . . . . . 33
Comunicacion . . . . . 33
Drivers . . . . . 34
Drivers para motor de tracción . . . . . 35
Driver motor torreta . . . . . 35
5.4.2. Conclusiones del Análisis de Implementación . . . . . 35

**6. Simulador** **37**
6.1. Entorno de Simulación . . . . . 37

**7. Pruebas y resultados** **39**
7.1. Pruebas en simulación . . . . . 39
7.1.1. Estructura de documentación de pruebas . . . . . 39
7.1.2. Calidad de mapeo con LiDAR simulado . . . . . 39
Descripción de la prueba . . . . . 39
Criterios de aprobación . . . . . 40
Procedimiento de ejecución . . . . . 40
Resultados . . . . . 40
7.1.3. Planificación y seguimiento de trayectoria en simulación . . 41
Descripción de la prueba . . . . . 41
Criterios de aprobación . . . . . 42
Procedimiento de ejecución . . . . . 42
Resultados . . . . . 43
7.1.4. Localización global bajo ruido de medición simulado . . . . 44
Descripción de la prueba . . . . . 44
Criterios de aprobación . . . . . 44
Procedimiento de ejecución . . . . . 45
Resultados . . . . . 45
7.1.5. Conmutación de estado FSM ante evento dinámico . . . . . 46
Descripción de la prueba . . . . . 46
Criterios de aprobación . . . . . 47
Procedimiento de ejecución . . . . . 47
Resultados . . . . . 48
7.2. Pruebas sobre prototipo físico . . . . . 48
7.2.1. Movilidad y capacidad de superación de terreno del prototipo 48
Descripción de la prueba . . . . . 48
Criterios de aprobación . . . . . 49
Procedimiento de ejecución . . . . . 49

Resultados . . . . . 49
7.2.2. Desempeño cinemático de la torreta (cañón) del prototipo . 50
Descripción de la prueba . . . . . 50
Criterios de aprobación . . . . . 50
Procedimiento de ejecución . . . . . 50
Resultados . . . . . 51
7.2.3. Detección y seguimiento de objetivo aéreo en prototipo físico 51
Descripción de la prueba . . . . . 51
Criterios de aprobación . . . . . 51
Procedimiento de ejecución . . . . . 51
Resultados . . . . . 52
7.2.4. Capacidad de disparo sobre objetivo estacionario y en movimiento . . . . . 52
Descripción de la prueba . . . . . 52
Criterios de aprobación . . . . . 52
Procedimiento de ejecución . . . . . 53
Resultados . . . . . 53
7.2.5. Trackeo múltiple por color y neutralización de blancos tipo globo . . . . . 53
Descripción de la prueba . . . . . 53
Criterios de aprobación . . . . . 54
Procedimiento de ejecución . . . . . 54
Resultados . . . . . 54

**8. Conclusiones** **55**
8.1. Conclusiones generales . . . . . 55
8.2. Próximos Pasos . . . . . 55

**A. Anexo A: Fuentes del análisis de mercado** **57**

**Bibliografía** **59**

# Índice de figuras

2.1. Ejemplo de seguimiento por color. . . . . 6
2.2. Ilustración del proceso de mapeo con LIDAR y SLAM. . . . . 7
2.3. Ejemplo de planificación de ruta utilizando el algoritmo A*. . . . . 9

5.1. Datos de las ruedas elegidas . . . . . 20
5.2. Pinout del Arduino nano . . . . . 21
5.3. Corte del eje . . . . . 23
5.4. Largueros del chasis . . . . . 24
5.5. Base del robot . . . . . 24
5.6. Coraza superior . . . . . 25
5.7. Corte lateral del robot . . . . . 26
5.8. Robot completo . . . . . 26
5.9. Máquina de estados finitos del prototipo . . . . . 29
5.10. Trama del protocolo . . . . . 33

6.1. Captura del entorno de simulación en Unreal Engine 5. . . . . 38

7.1. Comparación entre mapa de referencia del escenario y mapa de ocupación 2D generado por el módulo SLAM (celda de 0,15 m). . . 41
7.2. Evolución del índice IoU durante el barrido del entorno. . . . . 41
7.3. Trayectoria planificada y trayectoria ejecutada en el escenario de misión. . . . . 43
7.4. Error lateral respecto del camino planificado en función de la distancia recorrida. . . . . 44
7.5. Trayectoria de referencia y trayectoria estimada bajo ruido de medición global. . . . . 46
7.6. Error de posicionamiento global en función del tiempo de misión. . 46

# Índice de tablas

4.1. Estimación de costos de componentes principales del prototipo. . . 17

5.1. Mensajes, códigos y uso del payload . . . . . 34

7.1. Criterios de aprobación de la prueba de mapeo con LiDAR simulado. 40
7.2. Resultados medidos de la prueba de mapeo con LiDAR simulado. . 40
7.3. Criterios de aprobación de la prueba de planificación y seguimiento de trayectoria. . . . . 42
7.4. Resultados medidos de la prueba de planificación y seguimiento de trayectoria. . . . . 43
7.5. Criterios de aprobación de la prueba de localización global bajo ruido de medición. . . . . 44
7.6. Resultados medidos de la prueba de localización global bajo ruido de medición. . . . . 45
7.7. Criterios de aprobación de la prueba de conmutación FSM ante evento dinámico. . . . . 47
7.8. Resultados medidos de la prueba de conmutación FSM ante evento dinámico. . . . . 48
7.9. Criterios de aprobación de la prueba de movilidad del prototipo. . 49
7.10. Resultados medidos de la prueba de movilidad del prototipo. . . . 49
7.11. Criterios de aprobación de la prueba cinemática de torreta. . . . . 50
7.12. Resultados medidos de la prueba cinemática de torreta. . . . . 51
7.13. Criterios de aprobación de la prueba de detección y seguimiento. . 51
7.14. Resultados medidos de la prueba de detección y seguimiento. . . . 52
7.15. Criterios de aprobación de la prueba de disparo. . . . . 52
7.16. Resultados medidos de la prueba de disparo. . . . . 53
7.17. Criterios de aprobación de la prueba de trackeo múltiple y disparo. 54
7.18. Resultados medidos de la prueba de trackeo múltiple y disparo. . . 54

# Capítulo 1

# Presentación

## 1.1. Introducción

En los últimos años, los drones han emergido como herramientas clave en los conflictos bélicos modernos, representando una evolución significativa en la tecnología militar. El conflicto armado entre Rusia y Ucrania, iniciado en 2022 y sostenido hasta la actualidad, constituye el escenario más documentado de esta transformación: en ese frente los vehículos aéreos no tripulados pasaron de un rol complementario a un componente central del combate, con empleo masivo en reconocimiento, ataques de precisión mediante drones FPV y, de forma más reciente, plataformas guiadas por fibra óptica capaces de eludir las contramedidas electrónicas. Esta escalada pone de manifiesto la precisión, versatilidad y bajo costo relativo de la amenaza, al tiempo que expone las limitaciones de los sistemas de defensa actuales, que no logran neutralizarla de manera consistente.

La principal debilidad de las defensas convencionales radica en su incapacidad para contrarrestar la precisión y la adaptabilidad de los drones, así como en el elevado costo asociado al despliegue y mantenimiento de sistemas de misiles antiaéreos. En este contexto, surge la necesidad imperiosa de desarrollar soluciones innovadoras que permitan identificar y neutralizar estas amenazas de manera eficiente y económica.

El presente trabajo aborda esta problemática mediante la integración de técnicas avanzadas de visión computacional y aprendizaje automático en un robot militar, diseñado para operar en entornos hostiles. Este enfoque busca optimizar la detección y eliminación de objetivos, ofreciendo una alternativa robusta y efectiva a los sistemas de defensa tradicionales.

## 1.2. Antecedentes

Los drones comerciales comenzaron a utilizarse a gran escala en el conflicto Ruso-Ucraniano en el año 2022, marcando un punto de inflexión en el uso de tecnología accesible y versátil en el campo de batalla. Este fenómeno no solo subraya la capacidad de los drones para adaptarse rápidamente a las necesidades militares, sino que también refleja una tendencia histórica en la evolución de los sistemas de armas y defensa.

La historia de la guerra es, en esencia, una constante evolución entre las herramientas ofensivas y los mecanismos defensivos. Durante la Edad Media, los castillos representaban una defensa formidable contra los ejércitos a pie, proporcionando refugio y ventajas estratégicas. Sin embargo, la invención de la pólvora y

el desarrollo de cañones y armas de fuego hicieron obsoletos tanto a los castillos como a las armaduras tradicionales, redefiniendo las estrategias de combate.

De manera similar, los avances en los misiles antiaéreos impulsaron la evolución de los aviones de combate, dando lugar a modelos como el SR-71 Blackbird, diseñado para superar los sistemas de defensa convencionales mediante su velocidad y altitud extremas. Ahora, los drones han emergido como una alternativa altamente económica y fácil de desplegar, con la capacidad de infligir daño localizado tanto a personal como a vehículos. Esta nueva generación de tecnología bélica permite a los combatientes alcanzar objetivos estratégicos con precisión y a un costo significativamente menor en comparación con los sistemas tradicionales.

El conflicto en Ucrania ha puesto de manifiesto estas tendencias, demostrando cómo los drones pueden ser utilizados tanto para vigilancia como para ataques precisos, consolidando su posición como una herramienta clave en los conflictos modernos.

## 1.3. Estado del Arte

En la actualidad, los drones han transformado profundamente la dinámica de los conflictos bélicos, como se ha evidenciado en la guerra Ruso-Ucraniana. En los primeros días del conflicto, una de las estrategias defensivas más comunes contra los drones operados remotamente por pilotos consistía en que los soldados dispararan directamente hacia ellos. Sin embargo, esta táctica demostró ser poco efectiva, con tasas de acierto extremadamente bajas debido a la velocidad y maniobrabilidad de los drones.

Para mitigar los daños ocasionados por estos dispositivos, los vehículos comenzaron a incluir mayores niveles de armadura. Este enfoque condujo al renacimiento del concepto del "tanque tortuga", el cual consiste en la adición de armadura separada del casco principal del tanque. Este diseño busca disipar las explosiones generadas por los drones, protegiendo las partes más expuestas del vehículo.

Históricamente, los tanques como el T-72, de diseño ruso, y los modelos modernos americanos, como el M1 Abrams y el M2 Bradley, fueron concebidos principalmente para enfrentamientos directos contra otros tanques, en un contexto de posible guerra entre Rusia y Estados Unidos. Alternativamente, también se diseñaron para conflictos de çontrainsurgencia", combates contra fuerzas irregulares de países con recursos limitados. No obstante, la guerra a gran escala (LSCO, por sus siglas en inglés, "Large Scale Conflict") que se está desarrollando en Ucrania ha demostrado que estos diseños enfrentan desafíos significativos frente a las tácticas modernas que emplean drones.

Una característica clave de los drones empleados en el conflicto es que operan a través del espectro radioeléctrico, utilizando frecuencias de 390-510 MHz y 750-1050 MHz. Como contramedida, se han desplegado "jammers.º bloqueadores de señal para inutilizar las comunicaciones entre los drones y sus operadores. Sin embargo, esta medida ha llevado a una carrera armamentista tecnológica, donde nuevos protocolos de comunicación se desarrollan continuamente para permitir que los drones puedan evadir estas interferencias.

En este contexto, una innovación destacable ha sido la implementación de drones rusos controlados mediante fibra óptica. Esta tecnología los hace completamente

inmunes a los bloqueadores de señal, ya que elimina la dependencia del espectro radioeléctrico. Este avance representa un salto tecnológico significativo en el uso de drones en el campo de batalla, consolidando su papel como un componente esencial en las estrategias modernas de guerra.

## 1.4. Áreas Profesionales de Relevancia

- Ingeniería Electrónica
- Visión Computacional
- Inteligencia Artificial y Aprendizaje Automático
- Ingeniería Mecánica
- Ingeniería de Telecomunicaciones
- Diseño de Sistemas Embebidos
- Fabricación y Prototipado

## 1.5. Alcance y objetivos

En esta sección se definen los alcances y objetivos del proyecto, estableciendo el marco de trabajo y las metas específicas que guían el desarrollo del sistema robótico militar autónomo. Los objetivos se estructuran en un objetivo general y objetivos específicos, mientras que los alcances delimitan el ámbito de aplicación y las limitaciones del trabajo.

### 1.5.1. Objetivos generales

- **Desarrollar un sistema de entrenamiento virtual:** Este objetivo implica la creación de un sistema completo de entrenamiento virtual que integre herramientas de simulación avanzadas y algoritmos de aprendizaje automático. Se busca no solo diseñar escenarios virtuales realistas, sino también implementar un entorno interactivo que permita al robot militar aprender y adaptarse a diferentes situaciones de combate simuladas.
- **Construir un prototipo físico del robot militar:** Desarrollar un prototipo físico del robot a partir de parámetros previamente validados e iterados en el entorno virtual de simulación. En ese entorno se ajustan magnitudes que condicionan el diseño del hardware, como la apertura y la tasa de cuadros de la cámara, las velocidades de los motores de la torreta y del mecanismo de disparo, así como las calibraciones iniciales de los controladores PID de seguimiento y de los instrumentos de navegación, antes de trasladarlas al diseño, ensamblaje y validación del prototipo. Este enfoque permite que la plataforma física complemente y confirme los resultados obtenidos en simulación mediante pruebas en condiciones controladas, reduciendo iteraciones costosas sobre hardware real.
- **Integrar simulación virtual con validación física para garantizar robustez:** Combina la simulación virtual con la validación física para asegurar la robustez y efectividad del sistema en situaciones reales de combate. Este

enfoque integral garantiza que el robot esté completamente preparado para enfrentar los desafíos del campo de batalla, al haber sido entrenado y validado tanto en entornos virtuales como físicos.

- **Describir un proceso de producción para un prototipo productivo:** Este objetivo implica detallar las etapas necesarias para la producción de un prototipo funcional. Incluye definir el diseño preliminar, seleccionar componentes, integrar subsistemas de hardware y software, desarrollar e implementar algoritmos para navegación y control, ensamblar el prototipo con componentes electrónicos y mecánicos.

### 1.5.2. Objetivos particulares

- **Implementar algoritmos de navegación:** Crear algoritmos avanzados que permitan al robot moverse de manera autónoma y eficiente en diferentes entornos, asegurando su capacidad para evitar obstáculos y alcanzar objetivos específicos.
- **Utilizar técnicas de IA para detectar objetivos:** Aplicar algoritmos de inteligencia artificial que permitan al robot identificar y localizar objetivos de manera precisa, optimizando su capacidad de respuesta en tiempo real.
- **Integrar sensores y actuadores en el prototipo:** Incorporar y coordinar diversos sensores y actuadores en el prototipo del robot, permitiéndole interactuar de manera precisa con su entorno y recolectar datos esenciales para su funcionamiento y control.
- **Realizar pruebas y validación en entornos simulados:** Desarrollar y ejecutar un programa de pruebas en el entorno virtual (Unreal Engine 5) para verificar, a alto nivel, el desempeño de la navegación autónoma (localización global, mapeo con LiDAR simulado y planificación de trayectorias), la respuesta del software de control ante eventos dinámicos de misión y la coherencia de la máquina de estados frente a escenarios cambiantes. Las métricas obtenidas en simulación guían la calibración de parámetros y la iteración de algoritmos antes de trasladarlos al prototipo físico.
- **Realizar pruebas y validación en el prototipo físico:** Llevar a cabo ensayos sobre el prototipo en condiciones controladas para evaluar, a alto nivel, la movilidad de la plataforma, el desempeño cinemático de la torreta, la capacidad de seguimiento de objetivos mediante visión computacional y la efectividad de neutralización, incluyendo escenarios con uno o más blancos en movimiento. Se verifica además la integración operativa de sensores, actuadores y subsistemas de navegación. Como objetivo de misión integrado, se busca validar la detección y eliminación de objetivos de aproximadamente $45 \times 45$ cm a una distancia de 20 m.

# Capítulo 2

# Introducción Teórica

Este capítulo presenta los fundamentos teóricos que sustentan el desarrollo del sistema robótico militar autónomo. Se abordan los conceptos clave de cada tecnología implementada, desde la detección de imágenes mediante aprendizaje automático hasta los algoritmos de navegación y control, proporcionando el marco conceptual necesario para comprender las decisiones de diseño y la implementación del sistema.

## 2.1. Detección de imágenes

La detección de imágenes es un componente crucial en sistemas autónomos para la percepción del entorno y la identificación de objetos de interés. En el contexto de este proyecto, se centra en la detección específica de vehículos aéreos no tripulados (drones) hostiles. Para ello, se exploran algoritmos avanzados de visión computacional, capaces de procesar el flujo de video en tiempo real proveniente de las cámaras del robot.

Desde un punto de vista teórico, la detección de objetos se enmarca dentro del campo del aprendizaje profundo (*deep learning*), particularmente mediante el uso de redes neuronales convolucionales (CNN, por sus siglas en inglés: Convolutional Neural Networks). Las CNN han demostrado un desempeño sobresaliente en tareas de visión computacional debido a su capacidad para aprender características jerárquicas directamente de los datos de imagen, eliminando la necesidad de ingeniería manual de características. Modelos como YOLO (You Only Look Once) [1] y Faster R-CNN [2] son ejemplos prominentes en la detección de objetos en tiempo real. YOLO, por ejemplo, reformula la detección como un problema de regresión, prediciendo simultáneamente las cajas delimitadoras (*bounding boxes*) y las probabilidades de clase en una sola pasada de la red, lo que lo hace adecuado para aplicaciones donde la velocidad es crítica, como en este proyecto.

Adicionalmente, el sistema incorpora una funcionalidad de seguimiento por color (*color tracking*). Si bien la detección primaria de objetivos se basa en modelos más complejos, el seguimiento por color resulta útil para tareas de *troubleshooting* y calibración, y también para el seguimiento de blancos cooperativos. Se entiende por blanco cooperativo a un objetivo preparado de forma deliberada para facilitar su trackeo: incorpora una apariencia cromática distintiva y predecible (por ejemplo, un globo o marcador de color uniforme) que permite aislarlo rápidamente del entorno mediante segmentación, a diferencia de amenazas no cooperativas cuya apariencia no está adaptada a la detección. Este enfoque se basa en técnicas de procesamiento de imágenes, como la segmentación en el espacio de color HSV (Hue, Saturation, Value), que permite aislar regiones de interés basadas en

tonalidades específicas, resultando robusto frente a variaciones de iluminación en comparación con el espacio RGB.




FIGURA 2.1. Ejemplo de seguimiento por color.

El rendimiento y la robustez de los algoritmos de detección dependen en gran medida de la calidad y diversidad de los datos de entrenamiento. Por esta razón, se adoptó una estrategia híbrida para la construcción del conjunto de datos y el reentrenamiento de los modelos de visión, combinando:

- **Imágenes reales anotadas:** Se descargaron y utilizaron conjuntos públicos de Roboflow Universe con *bounding boxes* de drones en escenarios variados, complementados con capturas propias cuando fue necesario. Sobre estos datos se reentrenaron varios modelos de la familia YOLO mediante *fine-tuning* a partir de pesos preentrenados, ajustando la red de detección al dominio operativo del proyecto. Un ejemplo representativo de fuente empleada es el dataset *Drone Detection* [3].
- **Imágenes sintéticas:** Generadas en el entorno de simulación desarrollado en Unreal Engine 5, con variación de ángulos de visión, iluminación y modelos de aeronaves. Estas imágenes amplían la cobertura del entrenamiento y reducen la dependencia exclusiva de datos reales, costosos o de difícil acceso en un contexto de defensa.

Un aspecto crítico en el entrenamiento de modelos de aprendizaje profundo es el fenómeno del *overfitting*, donde el modelo aprende patrones específicos del conjunto de entrenamiento que no generalizan bien a datos nuevos. Para mitigarlo, se emplearon técnicas de regularización como *dropout*, aumento de datos (*data augmentation*, incluyendo transformaciones geométricas y de color aplicadas en el pipeline de entrenamiento de YOLO) y la incorporación de datos sintéticos, que enriquecen la variabilidad del conjunto.

En detección de drones, además, aparece un desbalance de clases: en escenas típicas predominan fondos sin objetivo y la clase *drone* representa una fracción minoritaria de los píxeles y anotaciones. Este problema se abordó en tres niveles del pipeline. En el **entrenamiento**, se aplicó aumento de datos agresivo sobre imágenes que contienen drones, muestreo orientado a ejemplos positivos y métricas de

evaluación por clase (precisión, *recall*, F1 y mAP) en lugar de depender de la exactitud global, que resulta engañosa ante clases desbalanceadas. En la **función de pérdida** del detector YOLO, la cabeza de clasificación pondera los errores sobre la clase de interés durante el reentrenamiento, de modo que falsos negativos en drones penalizan más el ajuste de pesos que los falsos positivos sobre fondo. En la **inferencia en el robot**, el módulo de seguimiento filtra detecciones por etiqueta de clase y umbral de confianza, seleccionando la caja delimitadora de mayor score antes de alimentar el control de torreta, descartando detecciones ambiguas o irrelevantes.

## 2.2. Mapeo

El mapeo es el proceso mediante el cual un robot construye una representación espacial del entorno que lo rodea. Esta representación es fundamental para la navegación autónoma, permitiendo al robot localizarse a sí mismo y planificar rutas hacia un objetivo. En este proyecto, el robot está equipado con un sensor LIDAR (Light Detection and Ranging), que proporciona mediciones precisas de distancia a los objetos circundantes en forma de una nube de puntos.

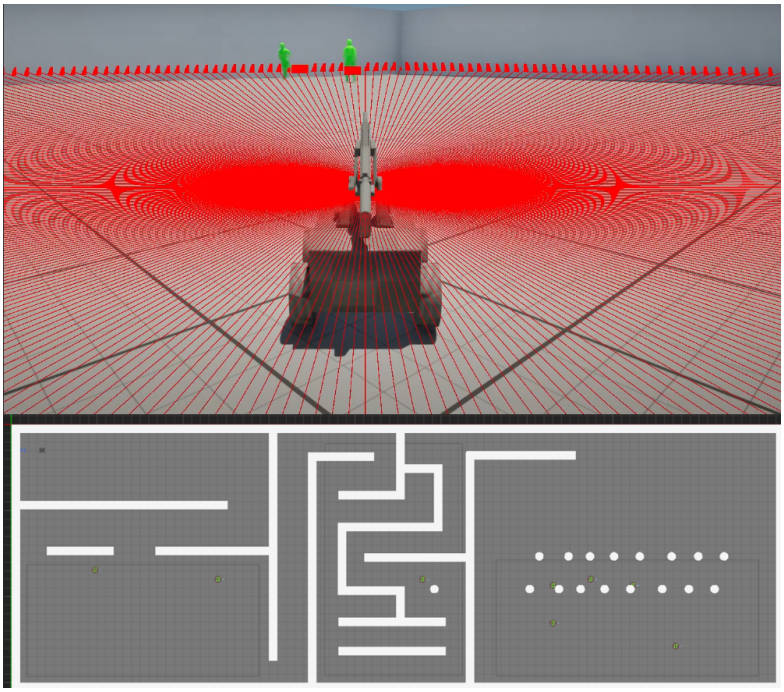


FIGURA 2.2. Ilustración del proceso de mapeo con LIDAR y SLAM.

Para construir el mapa a partir de los datos del LIDAR y, simultáneamente, estimar la pose (posición y orientación) del robot dentro de ese mapa, se recurre a técnicas de SLAM (Simultaneous Localization and Mapping) [4, 5]. El SLAM es un problema complejo, ya que la estimación de la pose del robot y la construcción del mapa son interdependientes. Desde un punto de vista teórico, el SLAM puede formularse como un problema de estimación bayesiana, donde se busca maximizar la probabilidad conjunta de la pose del robot y el mapa dado un conjunto de observaciones y controles [5]. Esto implica modelar la incertidumbre inherente

a las mediciones de los sensores y los errores acumulados en la odometría del robot.

Se investiga la aplicación de algoritmos eficientes de SLAM, con un enfoque particular en variantes como FastSLAM [6, 7], que descompone el problema para mejorar la eficiencia computacional, especialmente en entornos de gran escala. FastSLAM utiliza un filtro de partículas para representar la incertidumbre en la pose del robot y mantiene estimaciones separadas para los *landmarks* del entorno. Este enfoque combina las ventajas de los métodos basados en filtros de partículas con la eficiencia de los filtros de Kalman extendidos (EKF) para la estimación de *landmarks*, reduciendo la complejidad computacional de $O(N^2)$ a $O(N \log M)$, donde $N$ es el número de partículas y $M$ el número de *landmarks*. Además, se exploran enfoques más recientes como el SLAM basado en grafos (Graph-SLAM) [8, 9], que optimiza la trayectoria completa del robot y el mapa mediante la minimización de errores acumulados, siendo especialmente útil en entornos con bucles cerrados (*loop closure*).

Un desafío adicional en el contexto militar es la operación en entornos dinámicos, donde objetos o personas pueden moverse, invalidando partes del mapa. Esto requiere la implementación de estrategias de actualización dinámica del mapa y la detección de cambios en el entorno, un área activa de investigación en el campo del SLAM [9, 10].

## 2.3. Path planning

Una vez que el robot dispone de un mapa del entorno y conoce su ubicación, el siguiente paso es la planificación de rutas (*path planning*) [11]. Este proceso consiste en encontrar una trayectoria óptima o factible desde la posición actual del robot hasta un punto objetivo, evitando obstáculos y considerando otras restricciones (por ejemplo, el tipo de terreno o zonas prohibidas).

Para esta tarea, se implementa el algoritmo A* (A-star) [12, 13]. A* es un algoritmo de búsqueda informada que encuentra el camino de menor costo entre un nodo inicial y un nodo objetivo en un grafo. Utiliza una función heurística para estimar el costo restante hasta el objetivo, lo que guía la búsqueda hacia las regiones más prometedoras del espacio de estados. Es ampliamente utilizado en robótica debido a su completitud (garantiza encontrar una solución si existe) y optimalidad (encuentra el camino de menor costo si la heurística es admisible) [12]. El mapa generado por el módulo SLAM sirve como base para la construcción del grafo sobre el cual opera A*.

Desde un punto de vista teórico, A* pertenece a la familia de algoritmos de búsqueda heurística y se deriva de la búsqueda de Dijkstra [14], pero incorpora una estimación del costo futuro (heurística) para priorizar los nodos a explorar [12, 13]. La función de evaluación de A* se define como $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo acumulado desde el nodo inicial hasta el nodo $n$, y $h(n)$ es la heurística que estima el costo desde $n$ hasta el objetivo. Para garantizar la optimalidad, $h(n)$ debe ser admisible, es decir, nunca sobreestimar el costo real del camino restante [12, 13]. En este proyecto, se utiliza típicamente la distancia euclidiana o de Manhattan como heurística, dependiendo de la estructura del grafo que representa el mapa.

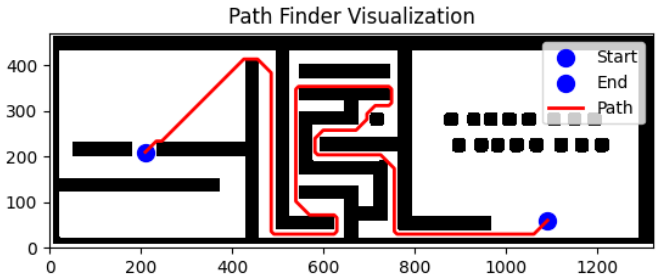



FIGURA 2.3. Ejemplo de planificación de ruta utilizando el algoritmo A*.

Además de A*, existen otros enfoques para la planificación de rutas, como los algoritmos probabilísticos, por ejemplo Rapidly-exploring Random Tree (RRT) [15, 11], que son más adecuados para espacios de alta dimensionalidad o entornos con obstáculos complejos. Sin embargo, A* se selecciona por su simplicidad de implementación y su eficacia en mapas discretizados, como los generados por SLAM. Un aspecto a considerar es la suavización de la trayectoria resultante, ya que A* puede generar caminos con cambios abruptos de dirección, no ideales para la dinámica de un robot físico. Para ello, se pueden aplicar técnicas de post-procesamiento como la interpolación de splines o algoritmos de suavizado basados en gradientes [11, 16].

## 2.4. Seguimiento de ruta

Tras la planificación de una ruta, el robot debe ser capaz de seguirla con precisión. El seguimiento de ruta, también conocido como control de trayectoria o *path tracking*, implica la implementación de algoritmos de control que ajustan continuamente los actuadores del robot (por ejemplo, velocidad y dirección de las ruedas o propulsores) para minimizar el error entre la posición actual del robot y la trayectoria deseada.

Existen diversas estrategias para el seguimiento de ruta, como los controladores proporcional-integral-derivativo (PID, por sus siglas en inglés: *Proportional-Integral-Derivative*) [17], algoritmos de *Pure Pursuit* [18], o técnicas basadas en modelos predictivos (MPC, *Model Predictive Control*) [19]. La elección del controlador adecuado depende de la dinámica del robot, las características del entorno y los requisitos de precisión de la misión. El objetivo es asegurar que el robot se mantenga sobre la ruta planificada por A*, incluso ante perturbaciones o errores de estimación en su localización.

Teóricamente, el problema del seguimiento de ruta puede formularse como un problema de control óptimo, donde se busca minimizar una función de costo que

representa el error de seguimiento y, potencialmente, otras variables como el consumo de energía o la suavidad del movimiento. Los controladores PID son una solución ampliamente utilizada debido a su simplicidad y robustez [17]. Un controlador PID calcula la acción de control como una combinación lineal de tres términos: el error proporcional (P), la integral del error acumulado (I) para eliminar errores estacionarios, y la derivada del error (D) para anticipar cambios en el error. Sin embargo, los PID requieren un ajuste cuidadoso de sus parámetros (ganancias $K_p$, $K_i$, $K_d$) para cada sistema específico, lo que puede ser un proceso iterativo.

Por otro lado, el algoritmo *Pure Pursuit* [18] es un método geométrico que calcula un punto de mira (*lookahead point*) en la trayectoria deseada, a una distancia fija del robot, y ajusta la dirección para alcanzarlo. Este enfoque es intuitivo y fácil de implementar, pero su desempeño depende críticamente de la elección de la distancia de anticipación, que debe balancear la estabilidad (distancias largas) con la precisión en curvas cerradas (distancias cortas). Finalmente, los controladores basados en modelos predictivos (MPC) [19] representan un enfoque más avanzado, donde se utiliza un modelo dinámico del robot para predecir su comportamiento futuro y optimizar las acciones de control en un horizonte de tiempo finito. Aunque computacionalmente más costoso, el MPC es capaz de manejar restricciones explícitas y dinámicas no lineales, siendo ideal para robots con comportamientos complejos.

## 2.5. Simulación

La simulación desempeña un papel fundamental en el desarrollo y validación de sistemas robóticos complejos, especialmente aquellos destinados a operar en entornos dinámicos o peligrosos. Permite probar algoritmos, evaluar diseños y realizar experimentos de forma segura, rápida y económica antes de realizar pruebas en el hardware físico.

Como se detalló en la Sección **??** , este proyecto utiliza Unreal Engine 5 (UE5) como plataforma de simulación. La elección de UE5 se justifica por su capacidad para generar entornos 3D realistas, simular físicas complejas y emular sensores como cámaras y LIDAR con alta fidelidad. Esto es esencial para:

- Validar los algoritmos de percepción (detección de imágenes, SLAM).
- Probar y refinar los algoritmos de planificación y seguimiento de ruta en diversos escenarios.
- Generar datos sintéticos para el entrenamiento de modelos de aprendizaje automático, como se mencionó anteriormente.
- Realizar pruebas de integración del sistema completo en un entorno controlado.

El uso de un simulador avanzado como UE5 acelera significativamente el ciclo de desarrollo, permite la exploración de un mayor número de escenarios y contribuye a la robustez general del sistema robótico.

Desde una perspectiva teórica, la simulación en robótica se basa en la creación de modelos matemáticos y computacionales que aproximan el comportamiento

del mundo real. Esto incluye la modelización de la dinámica del robot (cinemática y dinámica), las interacciones con el entorno (colisiones, fricción) y el ruido de los sensores. Un desafío clave es el *reality gap*, la discrepancia entre el comportamiento simulado y el real, que puede limitar la transferibilidad de los resultados de la simulación al hardware físico. Para minimizar este problema, UE5 permite la incorporación de ruido sintético en las mediciones de los sensores y la simulación de condiciones ambientales variables, acercando el entorno virtual al real. Además, técnicas como el aprendizaje por transferencia (*transfer learning*) y la adaptación al dominio (*domain adaptation*) se utilizan para ajustar los modelos entrenados en simulación a datos del mundo real.

# Capítulo 3

# Analisis técnico

## 3.1. sección 1

## 3.2. sección 2

# Capítulo 4

# Analisis de mercado

El presente análisis evalúa, en forma teórica, la posibilidad de inserción de un sistema robótico terrestre autónomo con capacidades de detección y neutralización de amenazas aéreas de baja cota (drones), con foco en su eventual presentación a las Fuerzas Armadas de la República Argentina.

## 4.1. Cliente objetivo primario en Argentina

El cliente objetivo primario es el Estado Nacional, en particular el sector defensa, ya que el producto se alinea con funciones de vigilancia, protección de objetivos críticos y apoyo a operaciones militares. Dentro de este marco, el **Fondo Nacional de la Defensa (FONDEF)** constituye un instrumento relevante para proyectos de reequipamiento y modernización.[1]

La Ley 27.565 establece que el FONDEF financia la recuperación, modernización e incorporación de material para las Fuerzas Armadas, y además fija criterios que favorecen.[2]

- Sustitución de importaciones y desarrollo de proveedores locales.
- Innovación tecnológica e industrial.
- Acciones de investigación y desarrollo en sector público y privado.

Estos criterios son consistentes con un desarrollo nacional que combine hardware, software, simulación y capacidades de integración.

## 4.2. Mercados potenciales para el producto

### 4.2.1. Sector público argentino

- **Defensa:** presentación de capacidad al Ministerio de Defensa y Fuerzas Armadas, con foco en vigilancia de instalaciones, protección de bases y apoyo a defensa de punto contra drones.
- **Seguridad interior:** fuerzas federales y organismos de seguridad para resguardo de infraestructura crítica y eventos de alta exposición.[3]

[1] Ley 27.565 (FONDEF), Boletín Oficial de la República Argentina, 2020.

[2] Ministerio de Defensa de la Nación, Fondo Nacional de la Defensa (FONDEF), sitio oficial.

[3] Decreto 21/2025, Boletín Oficial de la República Argentina, operación contractual para sistema antidrones.

En términos de canal formal de contratación, las adquisiciones estatales se publican y gestionan en la plataforma COMPR.AR,[4] con avisos vinculados en Boletín Oficial, incluyendo procesos de organismos militares.[5]

Como referencia comparada para compras de capacidades complejas de defensa, los procesos de NSPA (OTAN) muestran estructuras documentales con *Statement of Work*, matriz de evaluación técnica y requisitos de seguridad.[6]

En el plano regional, existen antecedentes de adquisiciones públicas de drones por fuerzas armadas, como la compra de drones de observación del Ejército de Chile.[7]

## 4.3. Análisis de costos del prototipo

Para esta etapa se estimó el costo de los componentes principales del prototipo físico. El análisis se limita al prototipo actual y no incluye costos de ingeniería, mano de obra, integración ni escalado productivo. Para el costo de energía de impresión 3D se considera un consumo típico de una impresora FDM de escritorio (aproximadamente 165 W de potencia media en operación continua, equivalente a $0,0164 ARS/s a tarifa residencial).

[4] COMPR.AR, Portal de Compras Públicas de la República Argentina.

[5] Boletín Oficial de la República Argentina, ejemplo de contratación de la Armada Argentina, Licitación Pública 0456/2025.

[6] NATO Support and Procurement Agency (NSPA), eProcurement, oportunidad de soporte C-UAS.

[7] ChileCompra, Licitación ID 3385-5-L124, adquisición de drones de observación por el Ejército de Chile.

| Componente | Cant. | Unitario (ARS) | Total (ARS) |
|---|---|---|---|
| Telémetro láser de largo alcance con interfaz digital y kit de envío internacional | 1 | 178.150 | 178.150 |
| Escáner LiDAR 2D de 360° para mapeo y navegación (tipo Slamtec A1M8) | 1 | 280.000 | 280.000 |
| Driver doble puente H para motores DC (tipo L298) | 2 | 3.332 | 6.664 |
| Módulo GNSS/GPS de posicionamiento con antena activa (tipo NEO-6M) | 1 | 12.141 | 12.141 |
| Marcadora neumática de baja energía para pruebas de puntería (plataforma airsoft) | 1 | 476.000 | 476.000 |
| Batería sellada de gel 12 V 7 Ah para tracción y electrónica | 2 | 30.000 | 60.000 |
| Conjunto de motores DC con reductora para tracción diferencial (kit x3) | 1 | 184.800 | 184.800 |
| Motor paso a paso NEMA 23 con reductor para eje de torreta | 1 | 138.600 | 138.600 |
| Microcontrolador de control embebido (formato Nano, MCU RP2040) | 1 | 150.000 | 150.000 |
| Cámara USB HD para adquisición de video en tiempo real | 1 | 29.000 | 29.000 |
| Kit de fijación mecánica (varillas roscadas, tuercas y arandelas) | 1 | 28.747 | 28.747 |
| Filamento PLA para fabricación aditiva (bobinas de 1 kg) | 7 | 18.000 | 126.000 |
| Rodamientos radiales sellados para ejes (pack x10, tipo 6202-2RS) | 1 | 24.000 | 24.000 |
| Consumo eléctrico de impresión 3D (40 h de operación, Ender 3) | 1 | 656 | 656 |
| **Total estimado del prototipo** | | | **1.694.758** |

TABLA 4.1. Estimación de costos de componentes principales del prototipo.

# Capítulo 5

# Prototipo

Al momento de fabricar el primer prototipo se optó por simplificar el diseño a modo de conseguir una POC funcional de las partes más innovadoras del sistema.

## 5.1. Selección de componentes

Se decidieron una serie de requerimientos simplificados para el prototipo a fin de poder seleccionar los componentes y el diseño adecuados.

- **Dimensiones:** 60cm x 60cm x60cm
- **Peso total:** 25Kg
- **Alcance efectivo:** 50m
- **Velocidad Máxima:** 3km/h

### 5.1.1. Ruedas

Se eligieron ruedas neumáticas 4.10/3.5-4 para 80 kg de carga, estas ruedas presentan buena tracción en terrenos difíciles y una buena capacidad de carga.




FIGURA 5.1. Datos de las ruedas elegidas

Al conocer el diámetro de las ruedas se pueden calcular el torque y las rpm de los motores.

### 5.1.2. motores de tracción

Para los motores de tracción de opto por motores PGM36-555, que son motores de 24V DC con reductor satelital, estos motores entregan 24Kgf/cm a 45rpm (58rpm en vacio) con un consumo de 0.7A, y un torque maximo de 80Kgf/cm con un consumo de 7.5A.

Con las ruedas previamente seleccionadas se puede calcular la velocidad robot

$$\begin{aligned} RPM &= 45 \\ D &= 0{,}25\,\mathrm{m} \\ C &= \pi \cdot D = \pi \cdot 0{,}25 \approx 0{,}7854\,\mathrm{m} \end{aligned}$$

Entonces la velocidad se calcula como:

$$v_{\mathrm{m/s}} = \frac{RPM \cdot C}{60} = \frac{45 \cdot 0{,}7854}{60} \approx 0{,}589\,\mathrm{m/s} \approx 2{,}12\,\mathrm{km/h}$$

### 5.1.3. Motores de la torreta

Para los dos ejes de la torreta se opto por motores stepper por su capacidad de mantener la posición al mantener energizadas las bobinas.

Para el eje X se opto por un motor tipo nema 17 de aproximadamente 2.6Kg/cm de torque de holding.

Para el eje Y, ya que este tiene que sostener el peso del arma se opto por un motor tipo nema 23 con reductor tipo sinfin y corona.

### 5.1.4. Drivers de motor

Una vez definidos los motores se buscaron los drivers para estos.

Para los motores de traccion se seleccionaron modulos puente H basados en BTS7960 ya que soportan tenciones entre 5.5V y 27V y corrientes de hasta 43A.

Para el motor nema 17 del eje X se opto por el driver A4988

El motor nema 23 del eje Y ya poseia su propio driver TB6600

### 5.1.5. Interfaz con la PC

Como el procesamiento sera realizado en la pc y la interfaz solo cumple la tarea de traducir las ordenes de esta a los drivers de motor se opto por una placa de desarrollo arduino nano, ya que esta cuenta con un puerto serie para conectarse a la pc y suficientes pines I/O para controlar todos los motores necesarios.

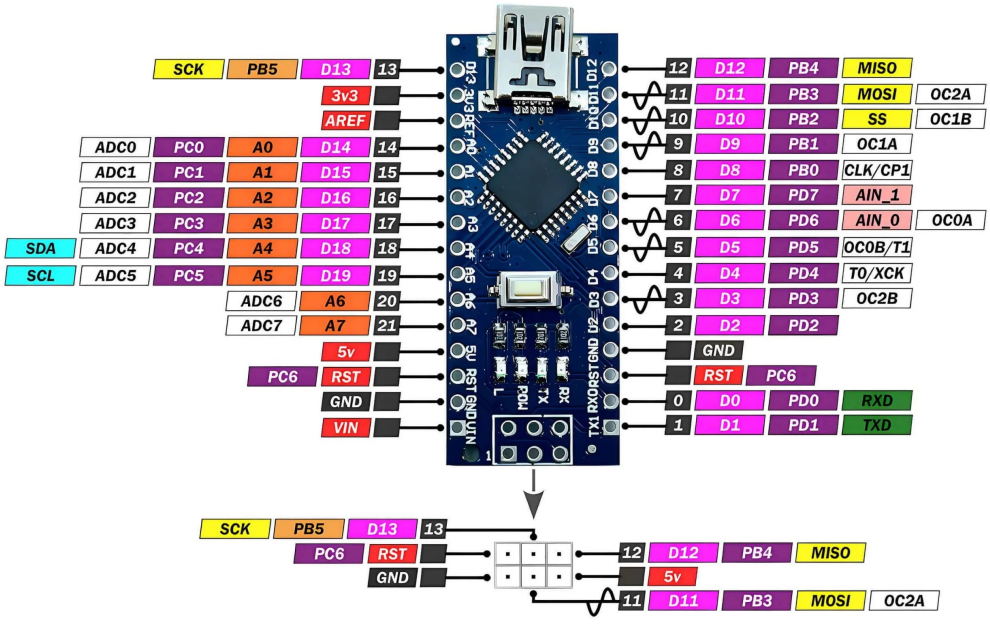



FIGURA 5.2. Pinout del Arduino nano

### 5.1.6. Baterias

Ya que los motores serian alimentados a 24v se opto por usar baterias recargables de gel de 12V y 7Ah puestas en serie para obtener los 24V

## 5.2. Chasis

Para facilitar la fabricación se opto por un chasis impreso en 3D para la POC.

Durante el proceso de diseño se tuvieron en cuenta las particularidades del proceso de impresión y los limites de volumen de las impresoras disponibles.

Con el objetivo de utilizar solo dos motores y mantener el movimiento solidario de las ruedas por lado se opto por unir los ejes delantero y trasero de cada lado con cadenas.

El diseño del chasis del robot se puede subdividir en secciones básicas que se unen entre si:

- **Ejes:** Soportan el peso del robot y transmiten la energía a las ruedas.
- **Largueros del chasis:** Proporcionan rigidez y junto con los ejes forman las unidades básicas de tracción del robot.
- **Pisos:** Terminan de conformar la base y proporcionan el lugar para almacenar baterías y la electrónica.
- **Coraza superior:** Da soporte a los componentes externos y la torreta.
- **Torreta:** Da soporte y permite apuntar el arma, telémetro y la cámara principal.

### 5.2.1. Análisis de Implementación

A continuación, se presenta un análisis detallado de las secciones básicas del chasis del robot y su implementación

#### Ejes

Los ejes tienen que soportar el peso del robot y al mismo tiempo transmitir la rotacion de los motores a las ruedas traseras y delanteras de cada lado de manera solidaria. Por esta razon se decidieron usar dos rulemanes de apoyo para soportar el peso de manera mas estable y se decidio agregar un piñon para unir las ruedas delanteras y traseras de cada lado con una cadena.

La estructura se compone de un eje central de metal, con los alojamientos para los dos rulemanes de apoyo, el piñón y el plato que sostiene la llanta, el cual tiene una chaveta para evitar que gire.

Los motores de tracción se fijan a los ejes con prisioneros que traban en el biselado de los ejes de los motores.

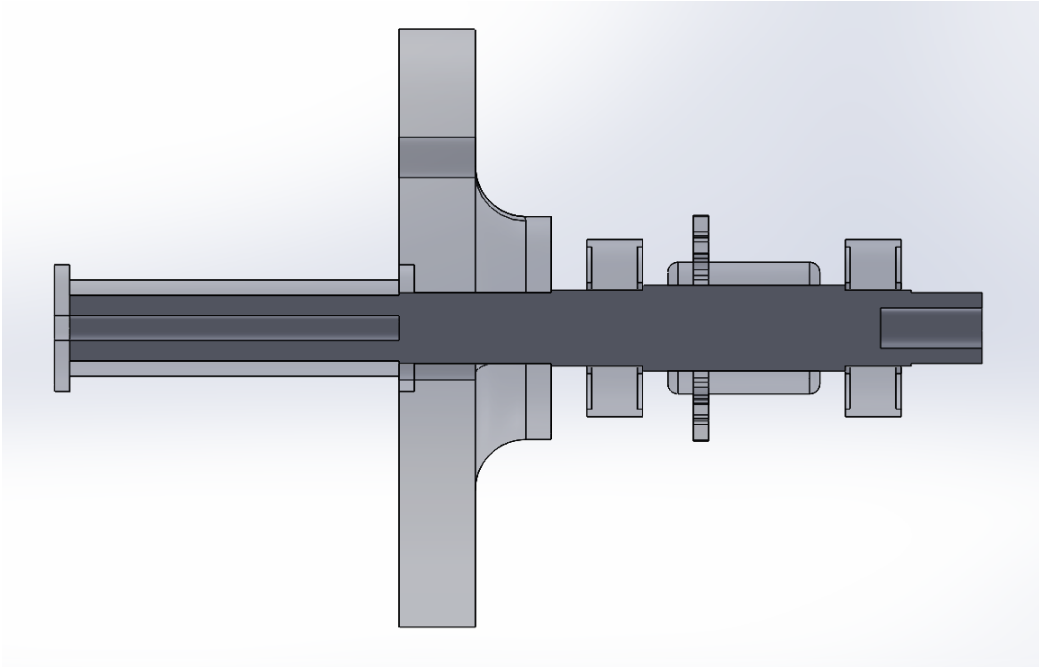


FIGURA 5.3. Corte del eje

**Largueros del chasis**

Los encargados de dar soporte a los ejes y mantener las cadenas de transmisión en su lugar son los largueros del chasis, los cuales son las piezas centrales en la estructura de la plataforma y junto con los motores y ruedas forman las unidades básicas de tracción del robot.

Se componen de 7 secciones, de las cuales 6 son la estructura del larguero y una ultima el soporte para el motor de tracción. En caso de cambiar a un esquema de 4 motores de tracción solamente es necesario agregar una placa de soporte extra delante ya que el diseño es simétrico.

Estas piezas deben ser ensambladas alrededor de los ejes de tracción ya que también limitan el movimiento axial de los rulemanes.

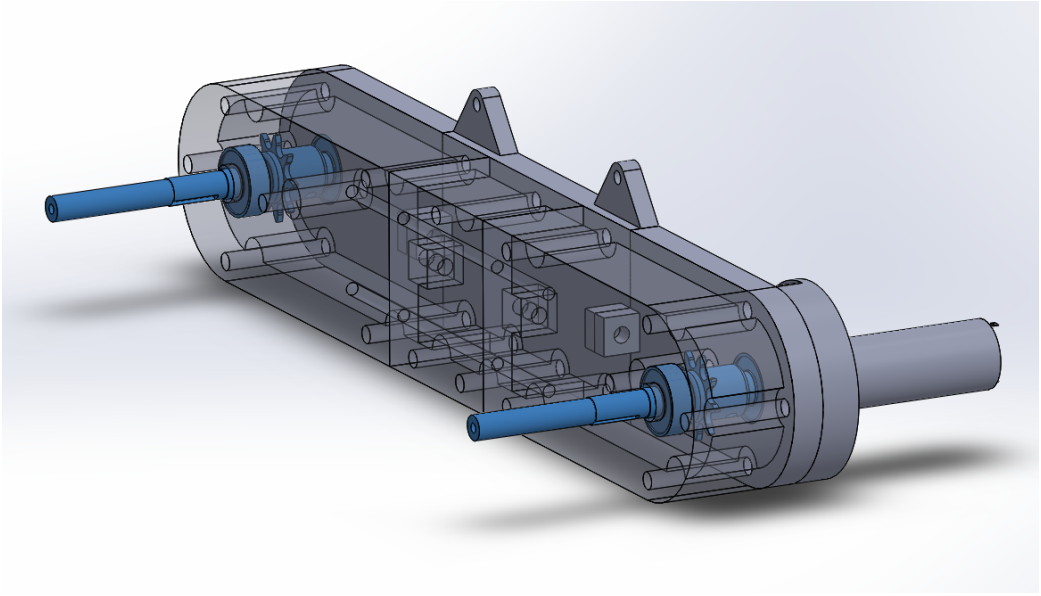


FIGURA 5.4. Largueros del chasis

**Pisos**

Los pisos se agregan a los largueros para terminar de conformar la estructura de la plataforma, la cual esta diseñada por secciones imprimibles que luego se unen entre si con varillas transversales para dar rigidez y fuerza al conjunto.

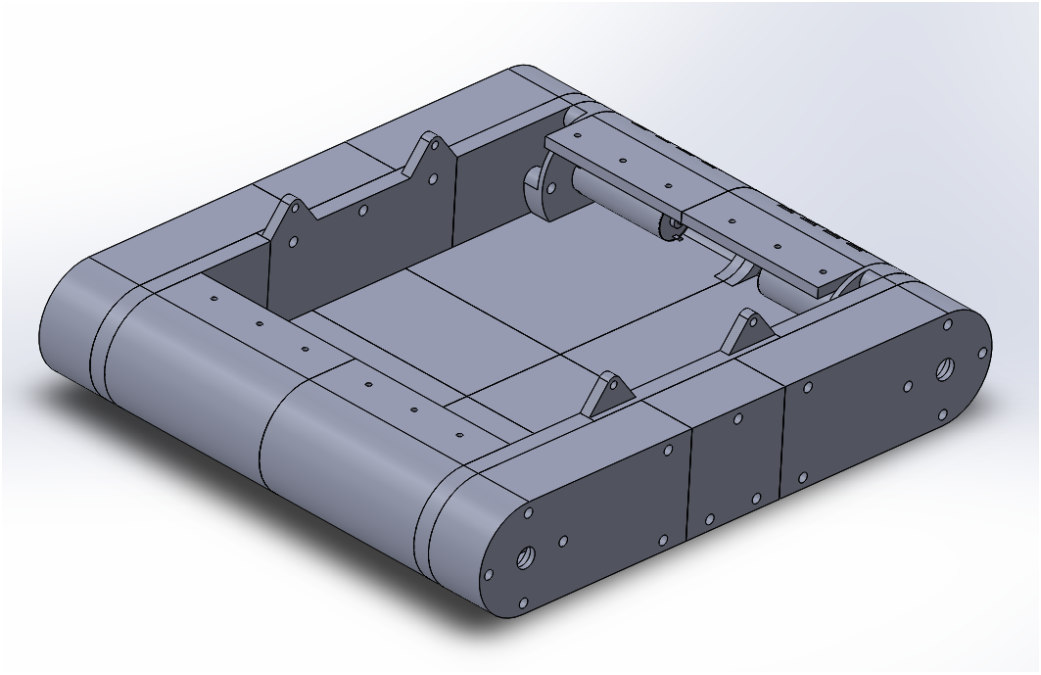


FIGURA 5.5. Base del robot

### Coraza superior

La coraza superior se diseño con forma de caparazón angulado, con una apertura para el sensor LIDAR y una plataforma para llevar la PC (que para simplificar las pruebas se dejo afuera del prototipo).

En el diseño de la coraza también se busco solucionar la limitación en el volumen de impresión por lo que se diseño una estructura con piezas que se intercalan para obtener rigidez, lo cual es esencial ya que la coraza da soporte a la torreta principal.

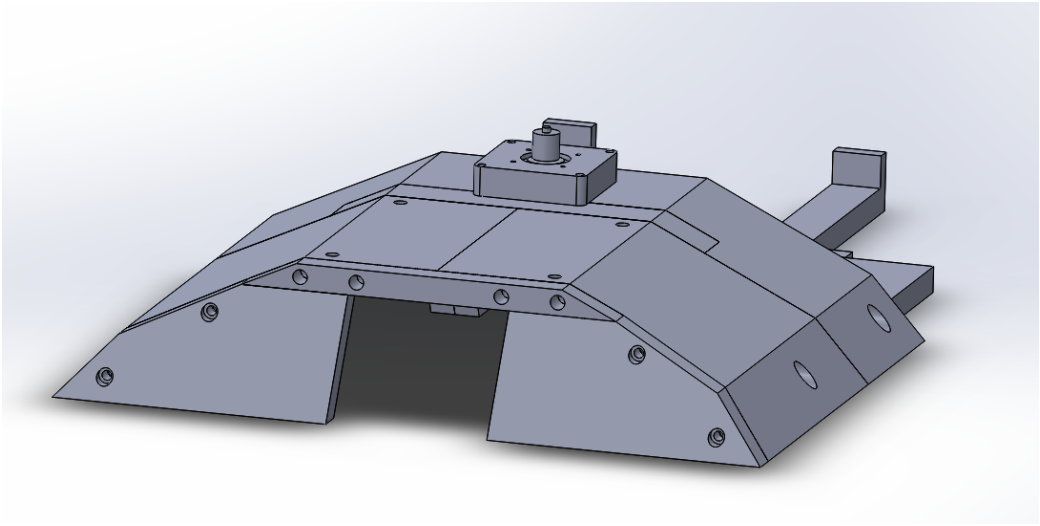


FIGURA 5.6. Coraza superior

### Torreta

La torreta se compone de una plataforma giratoria para el eje X que se conecta al motor con una correa y sobre esta plataforma se monto el soporte del motor y el arma con la articulación del eje Y.

El arma se agarra de un aplique fijo en su riel, en posición invertida.

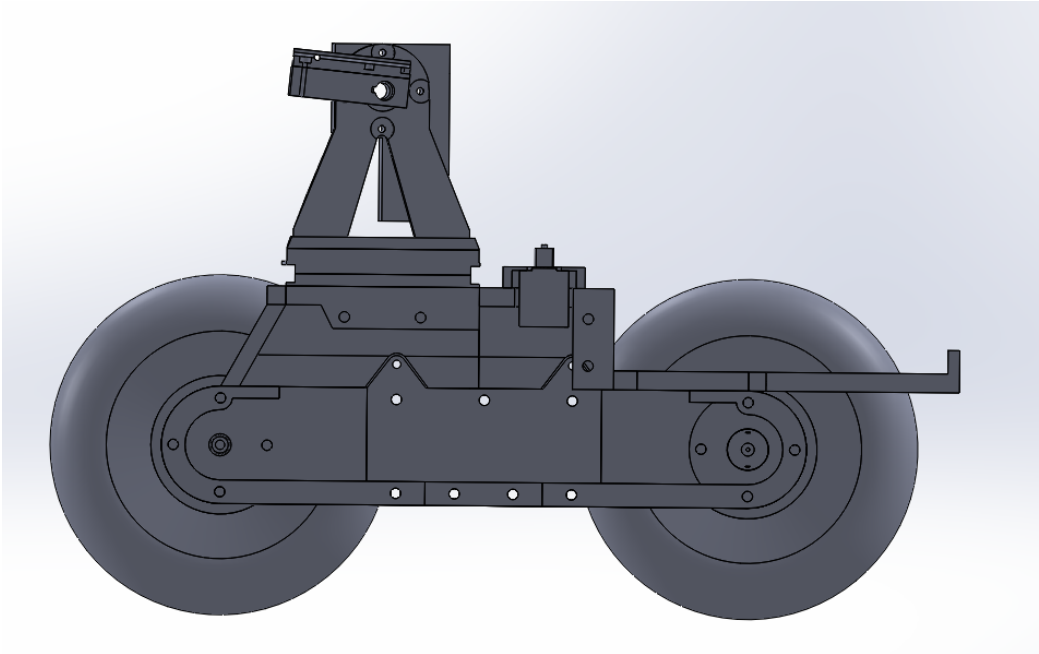


FIGURA 5.7. Corte lateral del robot

**Ensamblaje completo**

Las partes superior e inferior de la coraza se ensamblan con 4 encastres triangulares, lo cuales adicionalmente se aseguran con 4 tornillos y se conforma entonces la estructura completa del chasis del prototipo.

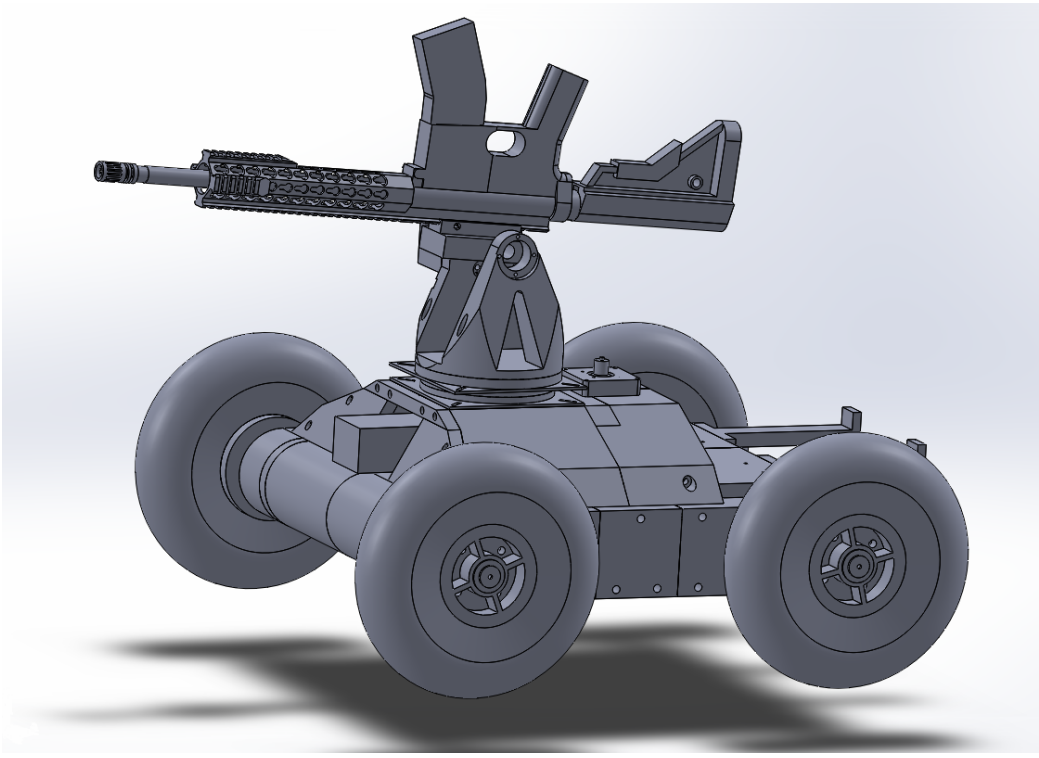


FIGURA 5.8. Robot completo

Para reducir tiempos y costos de prototipado se simularon por computadora las interacciones entre las piezas y la funcionalidad del prototipo lo que permitió que todas las partes funcionaran correctamente juntas antes de comenzar la impresión.

### 5.2.2. Conclusiones del Análisis de Implementación

El diseño del chasis se centro en los objetivos planteados para esta POC, se priorizo la facilidad de fabricación mediante impresión 3D y aseguro la modularidad y funcionalidad mecánica necesarias para el desplazamiento y operación del robot.

La estructura modular, segmentada en ejes, largueros, pisos, coraza y torreta, facilitó tanto el ensamblaje como la adaptación a cambios de diseño futuros.

Asimismo, se logró una integración estructural eficiente entre las piezas impresas, y un diseño que considera las limitaciones del volumen de impresión y maximiza la rigidez del conjunto. La simulación previa al prototipado físico permitió validar ensamblajes y detectar interferencias o problemas potenciales, y de esta manera reducir tiempos y costos en la fase de fabricación.

## 5.3. Software

El desarrollo del software del robot militar se realizó siguiendo el paradigma de Programación Orientada a Objetos (POO), permitiendo una estructura modular y escalable. Esta arquitectura facilita la separación clara de responsabilidades, el mantenimiento del código y la posibilidad de extender funcionalidades sin afectar el comportamiento general del sistema.

El sistema está dividido en tres módulos principales: control de misión, navegación autónoma y seguimiento de objetivos. Cada uno de estos módulos interactúa a través de interfaces bien definidas, lo que permite la sustitución o mejora de componentes sin necesidad de modificar el resto del sistema.

- **Control de misión:** Administra el estado general del robot y coordina los distintos subsistemas según el tipo de misión. Incluye lógica para transiciones entre estados como patrullaje, búsqueda de objetivos, eliminación y navegación hacia puntos estratégicos. También gestiona el tiempo de ejecución y las condiciones de activación de cada misión.
- **Seguimiento de objetivos:** Utiliza técnicas de visión computacional para analizar imágenes recibidas en tiempo real y detectar objetivos. Emplea controladores PID para alinear la torreta del robot con el objetivo detectado y evalúa condiciones de disparo según el nivel de precisión alcanzado.
- **Navegación autónoma:** Se basa en mapas de ocupación del entorno y algoritmos de planificación de trayectorias para determinar rutas seguras hacia los puntos de interés. Este módulo considera tanto la posición como la orientación del robot, y ajusta los comandos de movimiento para alcanzar los objetivos de forma precisa y eficiente.
- **Interfaces modulares:** El sistema fue diseñado para funcionar tanto en un entorno simulado como en uno real. Para ello, se implementaron drivers

abstractos que permiten controlar el robot mediante simuladores (como gamepads virtuales) o hardware real sin alterar la lógica del sistema principal. De igual manera, la comunicación entre módulos se realiza mediante un sistema de colas de mensajes, que asegura una interacción desacoplada y eficiente entre procesos.

Gracias a esta estructura modular y orientada a objetos, el software no solo puede adaptarse a distintos escenarios operativos, sino que también facilita futuras mejoras, pruebas automatizadas y escalabilidad hacia sistemas más complejos o entornos reales de combate.

### 5.3.1. Máquina de Estados Finitos

El comportamiento general del robot se organiza como una máquina de estados finitos (FSM, por sus siglas en inglés), lo que permite una lógica de control clara, robusta y fácilmente extensible. Cada estado representa una configuración operativa particular del robot, y las transiciones entre ellos se determinan en función de eventos y condiciones específicas del entorno o de la misión.

#### Estados Principales

- **STARTING:** Estado inicial del sistema. Evalúa el tipo de misión configurada y decide si comenzar en modo `SENTINEL` o `NAVIGATING`.
- **SENTINEL:** Modo de vigilancia estática en el que el robot escanea el entorno en busca de objetivos utilizando visión computacional. Si se detecta un objetivo, se transiciona al estado `TRACKING`.
- **TRACKING:** El robot ajusta la orientación de su torreta para seguir un objetivo en movimiento. Si logra un seguimiento preciso, transiciona al estado `TRACK_N_KILL`. Si pierde el objetivo, regresa a `SENTINEL` o `NAVIGATING`, dependiendo del estado anterior interrumpido.
- **TRACK_N_KILL:** Estado en el cual el robot ha fijado un blanco con alta precisión y activa su mecanismo de ataque. Si se pierde la fijación, vuelve a `TRACKING`.
- **NAVIGATING:** Estado de navegación autónoma hacia puntos estratégicos definidos en la misión. Si se detecta un objetivo durante la marcha, el sistema cambia a `TRACKING`. Al alcanzar un objetivo con tipo de misión de vigilancia, cambia a `SENTINEL`.
- **IDLE / PATROLLING / MAPPING:** Estados secundarios que permiten al robot detenerse, patrullar o explorar el entorno. Aunque no fueron parte del flujo principal en el prototipo básico, están contemplados para futuras funcionalidades.
- **DEBUG / ERROR:** Estados especiales para depuración o manejo de fallas críticas en tiempo de ejecución.

#### Transiciones

Las transiciones entre estados se determinan mediante condiciones lógicas evaluadas en tiempo real:

- **STARTING → SENTINEL/NAVIGATING:** Según si la misión es de vigilancia estática o de desplazamiento.
- **SENTINEL → TRACKING:** Activada cuando el sistema de visión detecta un objetivo.
- **TRACKING → TRACK_N_KILL:** Cuando el objetivo está centrado y estable.
- **TRACKING → SENTINEL/NAVIGATING:** Si se pierde el objetivo, se regresa al estado anterior.
- **TRACK_N_KILL → TRACKING:** Si se pierde la fijación sobre el objetivo.
- **NAVIGATING → TRACKING:** Si se detecta un objetivo durante el trayecto.
- **NAVIGATING → SENTINEL:** Cuando se alcanza el destino y el tipo de misión lo requiere.

Este enfoque basado en FSM proporciona una estructura robusta y predecible para la toma de decisiones del robot, facilitando además su validación mediante pruebas controladas y su extensión hacia escenarios más complejos.

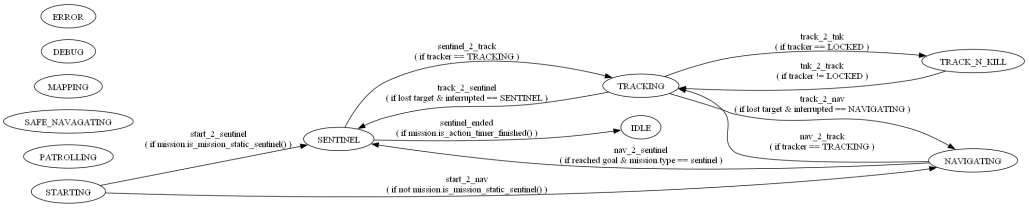



FIGURA 5.9. Máquina de estados finitos del prototipo

### 5.3.2. Análisis de Implementación

A continuación, se presenta un análisis detallado de la implementación del sistema, organizado en subsecciones que corresponden a los componentes clave del software. Este análisis se centra en el funcionamiento y el rol de cada componente dentro del sistema global, destacando los aspectos teóricos y las decisiones de diseño que sustentan su desarrollo. Cuando es relevante, se incluyen fragmentos de código para ilustrar conceptos específicos.

#### Control Central y Coordinación de Módulos

El componente central de control actúa como el núcleo del sistema robótico, encargado de coordinar las interacciones entre los módulos de seguimiento de objetivos y navegación autónoma, además de gestionar las transiciones de estado según la máquina de estados finitos descrita. Su rol principal es asegurar que el robot opere de manera coherente, alternando entre diferentes comportamientos según las condiciones de la misión y los eventos del entorno.

Desde un punto de vista teórico, este componente implementa un patrón de diseño de *mediador*, donde un elemento central coordina las interacciones entre múltiples subsistemas sin que estos se comuniquen directamente entre sí. Esto reduce el acoplamiento y mejora la modularidad, ya que los módulos solo interactúan

con el controlador central a través de interfaces bien definidas. Además, la capacidad de inicializar diferentes tipos de controladores para hardware virtual o real refleja el principio de *inversión de dependencias*, permitiendo que el sistema sea independiente de la implementación específica del hardware.

El bucle principal del sistema actualiza continuamente la posición y orientación del robot mediante mensajes recibidos de un sistema de colas de mensajería. Este enfoque de procesamiento basado en eventos es típico en sistemas reactivos, donde el robot debe responder en tiempo real a cambios en el entorno o a nuevas detecciones de objetivos. Un aspecto crítico del diseño es la gestión de interrupciones, como cuando se detecta un objetivo durante la navegación. El sistema guarda el estado anterior para permitir la reanudación de la tarea original una vez que se pierde el objetivo, lo que demuestra un manejo robusto de prioridades en un entorno dinámico.

**Seguimiento de Objetivos y Control de Torreta**

El módulo de seguimiento de objetivos es responsable de detectar y seguir objetivos mediante visión computacional, así como de controlar la torreta del robot para alinearla con el objetivo detectado. Su rol es crucial en las fases de vigilancia y ataque, permitiendo al robot identificar amenazas y actuar en consecuencia.

Este módulo utiliza un controlador PI (Proporcional-Integral), que combina términos proporcional e integral para minimizar el error entre la posición deseada y la actual de la torreta. Se optó por no incluir el término derivativo debido a que puede amplificar el ruido presente en las mediciones de la cámara y causar movimientos erráticos de la torreta. En la implementación, se observa cómo se limitan los valores integrales y de salida para evitar comportamientos inestables, como oscilaciones excesivas. A continuación, se muestra un fragmento de código que ilustra este control PI:

CÓDIGO 5.1. Control PI para seguimiento de objetivos

```
# Detectar objetivo en la imagen
objetivo = detectar_objetivo(imagen_camara)

# Calcular error de posicion respecto al centro
error_actual = calcular_error(objetivo, centro_pantalla)

# Aplicar control PI (sin termino derivativo)
error_proporcional = error_actual
error_integral += error_actual
salida_pi = Kp * error_proporcional + Ki * error_integral

# Limitar salida para evitar oscilaciones
salida_limitada = limitar_salida(salida_pi)

# Ajustar orientacion de la torreta
torreta.mover(salida_limitada)

# Si el objetivo esta centrado, activar disparo
if objetivo_centrado(error_actual):
    sistema_disparo.activar()
```

El módulo procesa datos de video recibidos a través de un sistema de mensajería, filtrando objetivos según una etiqueta específica y calculando el centro del cuadro delimitador del objetivo con mayor confianza. Luego, determina si el objetivo está dentro de un margen de seguridad para decidir si está en modo de bloqueo (listo para disparar) o de seguimiento activo. Este enfoque de umbral binario simplifica la decisión de disparo, aunque podría mejorarse con técnicas más avanzadas como filtros de Kalman para suavizar las trayectorias de seguimiento.

Además, cuando no hay objetivos detectados, el módulo implementa un comportamiento de búsqueda, moviendo la torreta en un patrón de barrido definido por ángulos máximos y mínimos. Este comportamiento refleja un enfoque de exploración sistemática, común en sistemas de vigilancia autónoma.

**Navegación Autónoma y Planificación de Rutas**

El módulo de navegación autónoma gestiona el desplazamiento del robot hacia puntos de interés definidos en una misión. Su rol es determinar rutas seguras y eficientes utilizando mapas de ocupación del entorno y algoritmos de planificación de trayectorias, asegurando que el robot alcance sus objetivos mientras evita obstáculos.

Desde una perspectiva teórica, la navegación autónoma integra localización, mapeo y planificación de rutas. Este módulo calcula rutas óptimas mediante el algoritmo A*, como se discutió en la sección teórica previa. Carga un mapa de ocupación y genera una ruta hacia el objetivo, almacenando los puntos intermedios (waypoints) para su seguimiento. El control de movimiento ajusta la velocidad lineal y angular del robot en función de la distancia y el ángulo hacia el objetivo, utilizando un enfoque proporcional simple para evitar sobrepasar el destino, como se muestra en el siguiente fragmento:

CÓDIGO 5.2. Control proporcional para navegación

```
# Calcular distancia al objetivo
distancia_al_objetivo = calcular_distancia(posicion_actual, objetivo)

# Control proporcional de velocidad
velocidad_adelante = distancia_al_objetivo * ganancia_velocidad
velocidad_adelante = limitar_velocidad(velocidad_adelante)

# Calcular error angular y ajustar giro
error_angular = calcular_error_orientacion(objetivo)
velocidad_giro = error_angular * ganancia_giro
velocidad_giro = limitar_giro(velocidad_giro)

# Mover el robot
robot.mover(velocidad_adelante, velocidad_giro)
```

Este método podría beneficiarse de controladores más sofisticados como PID o MPC (Model Predictive Control) para manejar dinámicas más complejas o terrenos irregulares. Un aspecto notable es el registro de la trayectoria del robot, que se guarda para análisis posterior y depuración, un aspecto crucial en el desarrollo de sistemas autónomos.

**Comunicación y Mensajería entre Módulos**

El sistema de comunicación basado en colas de mensajes es fundamental para la arquitectura desacoplada del software, permitiendo que los módulos intercambien datos como imágenes procesadas o información de posición sin una conexión directa. Su rol es asegurar una interacción eficiente y asíncrona entre los componentes del sistema.

Teóricamente, este sistema implementa el patrón de diseño *publish-subscribe* combinado con colas de trabajo, lo que asegura que los mensajes sean persistentes y procesados de manera ordenada. Permite a los módulos acceder a los datos más recientes o limpiar la cola para evitar acumulación de mensajes obsoletos, lo que es crítico en aplicaciones en tiempo real donde la latencia debe minimizarse. Este enfoque de comunicación asíncrona es ideal para sistemas distribuidos, ya que permite que los componentes operen de manera independiente y a diferentes velocidades de procesamiento. Sin embargo, introduce desafíos como la gestión de errores en la decodificación de mensajes y la posible pérdida de datos si no se implementan mecanismos de confirmación adecuados.

**Configuración y Modularidad del Sistema**

Un tema recurrente en la implementación es el uso de archivos de configuración para parametrizar el comportamiento del sistema. Esto se observa en la inicialización de todos los módulos principales, donde parámetros como ganancias de control, márgenes de seguridad y configuraciones de hardware se cargan dinámicamente. Desde un punto de vista teórico, este enfoque sigue el principio de *configuración sobre convención*, permitiendo ajustes sin modificar la lógica interna, lo que es esencial para pruebas y adaptaciones a diferentes entornos o hardware.

Además, la modularidad se refuerza mediante el uso de controladores virtuales que abstraen la interacción con el hardware. Esto refleja el patrón de diseño *adaptador*, facilitando la transición entre simulación y operación real, un aspecto clave en el desarrollo de sistemas robóticos donde las pruebas en hardware físico pueden ser costosas o peligrosas.

### 5.3.3. Conclusiones del Análisis de Implementación

El análisis de la implementación revela un diseño de software bien estructurado, basado en principios de modularidad, desacoplamiento y configurabilidad. La implementación de una máquina de estados finitos proporciona un marco robusto para la toma de decisiones, mientras que los módulos de seguimiento y navegación utilizan técnicas de control clásicas y modernas para cumplir con los objetivos de la misión. El sistema de comunicación asegura una interacción eficiente entre componentes, aunque podría beneficiarse de mecanismos adicionales de manejo de errores y sincronización.

En términos teóricos, el software refleja una combinación de enfoques reactivos y deliberativos: reactivos en la respuesta inmediata a detecciones de objetivos, y deliberativos en la planificación de rutas y gestión de misiones. Esta dualidad es típica en robótica autónoma, donde se requiere un equilibrio entre rapidez de respuesta y planificación a largo plazo. Futuras mejoras podrían incluir la integración de algoritmos de aprendizaje automático para la adaptación dinámica de

parámetros de control y la implementación de técnicas de fusión de sensores para mejorar la robustez en entornos complejos.

## 5.4. Interfaz y control de motores

El desarrollo de la interfaz entre la PC y los motores del robot se realizó priorizando funcionalidad, simplicidad y robustez. El sistema se diseñó para que la placa controladora actúe únicamente como una unidad de control de bajo nivel, recibiendo comandos desde la PC y ejecutando las órdenes directamente sobre los actuadores, sin necesidad de procesamiento complejo en el microcontrolador.

Este diseño modular permite además implementar distintos modos de operación sin modificar el hardware: control local desde una computadora de abordo, teleoperación remota o integración con sistemas autónomos, cambiando únicamente el medio físico de comunicación.

### 5.4.1. Análisis de Implementación

El sistema de control está dividido en tres capas software principales:

- **Capa de aplicación (PC):** Utiliza la clase RobotDriver, que actúa como interfaz de alto nivel (wrapper) para generar comandos de movimiento, torreta y disparo sin interactuar directamente con los frames del protocolo.
- **Capa de comunicación (PC–Microcontrolador):** Implementada en la PC en Python mediante RobotConnector, encargada de formar, enviar y validar tramas sobre el enlace UART. Y en el Microcontrolador en C++ mediante RobotListener, encargade de recepción, decodificación y validación de tramas.
- **Capa de control de hardware (Microcontrolador):** Implementa la clase Robot, encargada del control directo de motores DC, motores paso a paso e IO mediante interrupciones.

Dentro de la implementacion las partes mas destacables son la capa de comunicacion y la de control de hardware.

#### Comunicacion

La comunicación entre la PC y la placa se implementó sobre un enlace UART, utilizando un protocolo propio basado en tramas compactas de longitud variable.

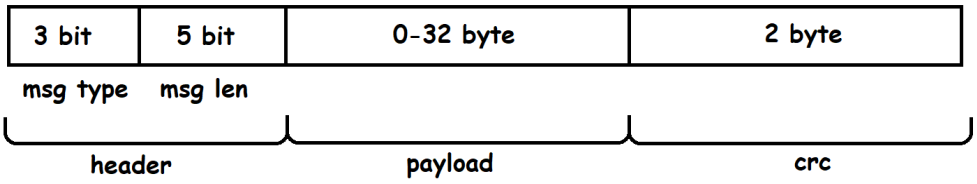



FIGURA 5.10. Trama del protocolo

El flujo completo de comunicación es el siguiente:

| Tipo | Código | Payload | Uso |
|---|---|---|---|
| ping | 0b000 | No | Verificación de enlace. |
| data | 0b001 | Sí | Envio de data. |
| angle | 0b010 | No | Consulta de posición de la torreta. |
| ack | 0b011 | No | Confirmación válida. |
| nak | 0b100 | No | Error de CRC, solicitar retransmisión. |
| home | 0b101 | No | Enviar torreta a origen. |
| metrics | 0b110 | No | Solicita datos del sistema. |
| stop | 0b111 | No | Detener actuadores. |

TABLA 5.1. Mensajes, códigos y uso del payload

1. **Generación de la trama en la PC:** Se construye la trama, incluyendo el encabezado, el payload y el CRC-16 correspondiente.
2. **Transmisión hacia el microcontrolador:** La trama completa se envía.
3. **Recepción y decodificación en la placa:** El microcontrolador recibe los bytes, interpreta el encabezado y separa el payload y el CRC recibido.
4. **Validación mediante CRC:** La placa calcula localmente el CRC-16 del header y payload recibidos y lo compara con el CRC transmitido:
   - Si los valores **coinciden**, la trama es válida: la placa ejecuta el comando solicitado y responde con una trama de tipo `ACK`.
   - Si los valores **no coinciden**, se considera error de transmisión: la placa responde con una trama `NAK`.
5. **Retransmisión automática en caso de error:** Si la PC recibe un `NAK`, interpreta que ocurrió un error de comunicación y retransmite automáticamente la última trama enviada, garantizando la entrega confiable del comando.
6. **Pedido de datos de telemetria:** Si el controlador recibe un frame de tipo `metrics`, entonces responde con una trama de tipo `data` con la información de las métricas, la respuesta de tipo `data` se interpreta como un `ACK`.

El CRC utilizado es CRC-16 con polinomio `0x1021` e inicialización `0xFFFF`, calculado sobre el header y el payload.

Ademas se implementa un timeout para la recepción de la confirmación, de esta manera se asegura un funcionamiento robusto del sistema incluso en presencia de ruido o pérdidas temporales en el enlace.

Para controlar el robot se usan principalmente tramas de tipo `data` compuestas de la siguiente manera:

```
[0b001101][velL, velR, velPitch, velYaw, shoot][CRC]
```

Las velocidades estan expresadas entre 0 y 255, siendo 0 velocidad totalmente negativa, 255 maxima velocidad positiva y 128 velocidad 0.

### Drivers

Para el control del robot se utilizaron dos tipos de actuadores:

- Motores DC con puente H, utilizados para tracción.
- Motores paso a paso, utilizados para la torreta.

Debido a las diferencias entre ambos actuadores, los controladores fueron diseñados de forma independiente.

#### Drivers para motor de tracción

Los puentes H que usan los motores de tracción izquierdo y derecho son controlados mediante tres señales, dos de dirección que controlan el sentido de giro y una de marcha en la que se utiliza PWM benedictino un control de la velocidad de giro de las ruedas.

Para las señales PWM se utilizo el timer0 del microcontrolador.

#### Driver motor torreta

Pese a que los drivers utilizados para los motores stepper de elevación y giro de la torreta son diferentes debido a las diferencias en requisitos de potencia de los motores usados, las señales de control que necesitan son iguales, por lo que a nivel lógico los controladores se pudieron implementar de igual manera. Ambos drivers requieren una señal digital de dirección y una señal digital cuyo flanco ascendente indica un paso. Se utilizo el timer1 del microcontrolador para disparar interrupciones, lo que genera un ciclo de control estable para actualizar la señal de control de velocidad y logra un movimiento fluido y controlado de los motores de la torreta.

### 5.4.2. Conclusiones del Análisis de Implementación

La estructura en tres capas principales que organizan el flujo de información desde la PC hasta los actuadores del robot separa claramente las responsabilidades de alto nivel, comunicación y control físico, lo que simplifica el diseño y facilita la expansión del sistema.

La implementación de un protocolo con confirmación de recepción y validación de tramas brinda robustez al sistema y el uso de los timers internos del micro permitió generar un ciclo de control estable para los actuadores independiente del tiempo de procesamiento de las tramas, lo que permite lograr mayores velocidades angulares en los motores stepper.

Si bien no se implemento la recopilación de datos de telemetria por parte del prototipo, el haberlos contemplado en el protocolo de comunicación hace que en desarrollos futuros sea posible agregar esta funcionalidad sin modificaciones sustanciales de software.

# Capítulo 6

# Simulador

Una de las bases del proyecto es la capacidad de poder validar el comportamiento del robot de manera dinámica y rápida en condiciones cambiantes. Esto, ademas de aumentar la robustez del sistema, permite que el mismo puede ser configurado específicamente para un tipo de misión o situación.

## 6.1. Entorno de Simulación

Para lograr este objetivo, se optó por desarrollar un entorno de simulación robusto y versátil utilizando el motor de videojuegos Unreal Engine 5. La elección de Unreal Engine 5 se fundamenta en varias de sus características clave que se alinean directamente con las necesidades del proyecto:

- **Realismo Visual y Físico:** Unreal Engine 5 ofrece capacidades gráficas de vanguardia y un motor de físicas avanzado. Esto permite recrear escenarios de operación con un alto grado de fidelidad, lo que es crucial para validar algoritmos de visión computacional y la interacción del robot con el entorno. Se pueden simular diversas condiciones de iluminación, terrenos y obstáculos que el robot podría encontrar en un despliegue real.
- **Flexibilidad y Extensibilidad:** El motor proporciona un amplio conjunto de herramientas y la posibilidad de C++ scripting, lo que permite una personalización profunda del entorno de simulación. Esto facilita la integración de modelos específicos del robot, sensores (como cámaras, LiDAR) y actuadores, así como la implementación de lógicas de comportamiento complejas para los objetivos y otros agentes en el simulador.
- **Generación de Datos Sintéticos:** Una ventaja significativa es la capacidad de generar grandes volúmenes de datos sintéticos etiquetados. Estos datos son invaluables para el entrenamiento y la validación de los modelos de aprendizaje automático utilizados en el sistema de identificación de objetivos, reduciendo la dependencia de la recolección de datos en el mundo real, que puede ser costosa y peligrosa en un contexto militar.
- **Iteración Rápida:** La naturaleza interactiva de un motor de videojuegos permite realizar pruebas y ajustes de manera ágil. Se pueden modificar parámetros del robot, del entorno o de la misión y observar los resultados de forma inmediata, acelerando el ciclo de desarrollo y depuración.
- **Soporte Comunitario y Documentación:** Unreal Engine cuenta con una vasta comunidad de desarrolladores y una extensa documentación, lo que

facilita la resolución de problemas y el aprendizaje de funcionalidades avanzadas.

El simulador desarrollado permite configurar diferentes escenarios de misión, variando elementos como la disposición de los objetivos, las características del terreno, la presencia de obstáculos y las condiciones ambientales. Esto no solo sirve para probar la robustez general del sistema de navegación y control del robot, sino también para evaluar la efectividad de las estrategias de identificación y eliminación de objetivos bajo diversas circunstancias.

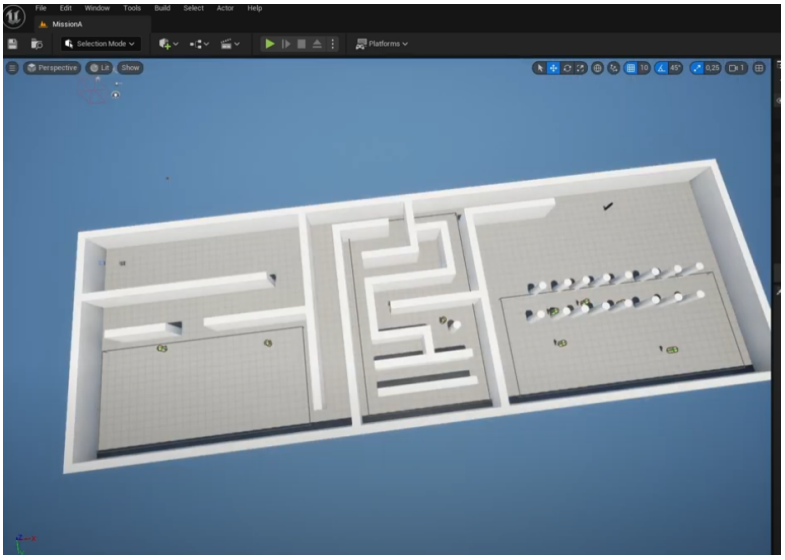


FIGURA 6.1. Captura del entorno de simulación en Unreal Engine 5.

En resumen, la utilización de Unreal Engine 5 como base para el simulador ha proporcionado una plataforma potente y adaptable, esencial para el desarrollo, prueba y validación iterativa del sistema robótico militar propuesto en esta tesis. Permite emular con precisión las condiciones operativas y evaluar el desempeño de los algoritmos de visión computacional y aprendizaje automático en un entorno seguro y controlado antes de su implementación en el prototipo físico.

# Capítulo 7

# Pruebas y resultados

En este capítulo se presentan las pruebas de validación del sistema, sus alcances y los resultados obtenidos en cada caso. Para mantener consistencia metodológica, cada prueba se documenta con la misma estructura: descripción, criterios de aprobación, procedimiento de ejecución y resultados.

Se incluyen ensayos en entorno virtual, orientados a validar navegación autónoma, mapeo con LiDAR simulado e interacción frente a eventos dinámicos, así como ensayos sobre el prototipo físico que verifican movilidad, desempeño de la torreta, seguimiento de objetivos y efectividad de neutralización.

## 7.1. Pruebas en simulación

Las pruebas en simulación se ejecutan en el entorno virtual desarrollado en Unreal Engine 5, con sensores y dinámica equivalentes al sistema propuesto. Esta instancia permite validar desempeño de navegación y percepción en condiciones controladas antes de la campaña experimental sobre hardware.

### 7.1.1. Estructura de documentación de pruebas

Todas las pruebas de este capítulo siguen la misma estructura:

- **Descripción de la prueba:** objetivo técnico y alcance.
- **Criterios de aprobación:** umbrales mínimos de desempeño.
- **Procedimiento de ejecución:** secuencia de pasos y parámetros de corrida.
- **Resultados:** métricas medidas y veredicto de cumplimiento.

### 7.1.2. Calidad de mapeo con LiDAR simulado

#### Descripción de la prueba

Esta prueba evalúa de forma aislada la capacidad del módulo SLAM para construir un mapa de ocupación preciso y completo a partir del LiDAR simulado, sin considerar la calidad de navegación ni eventos externos. El robot recorre un entorno virtual cerrado con geometría conocida, y el mapa generado se compara contra el mapa de referencia del escenario.

#### Objetivos mínimos

- Verificar que el mapa cubre la totalidad del área navegable del entorno.

- Validar la precisión geométrica del mapa respecto al mapa de referencia.
- Medir la velocidad de convergencia del mapa durante el barrido.
- Cuantificar errores de clasificación de celdas (falsas ocupaciones).

**Criterios de aprobación**

| Métrica | Criterio de aprobación |
|---|---|
| Calidad del mapa de ocupación (IoU mapa generado vs mapa de referencia) | $\geq 0{,}75$ |
| Cobertura del área navegable en el mapa generado | $\geq 90\,\%$ |
| Porcentaje de celdas libres clasificadas erróneamente como ocupadas | $\leq 5\,\%$ |
| Tiempo de convergencia del mapa (IoU $\geq 0{,}70$ estable por $\geq 10$ s) | $\leq 5$ min |
| Entropía media de la grilla al finalizar el barrido | $\leq 0{,}15$ bits/celda |

TABLA 7.1. Criterios de aprobación de la prueba de mapeo con LiDAR simulado.

**Procedimiento de ejecución**

1. Configurar escenario virtual cerrado con geometría conocida y exportar mapa de referencia en formato de grilla de ocupación 2D, con resolución de celda de $0{,}15$ m.
2. Inicializar el robot en posición de inicio conocida con módulo SLAM activo y LiDAR simulado habilitado.
3. Ejecutar un barrido manual o teleoperado del entorno completo, sin activar planificación de trayectoria autónoma.
4. Registrar el estado del mapa a intervalos de 10 s: IoU parcial, cobertura y entropía de la grilla.
5. Al finalizar el barrido, exportar el mapa generado y compararlo contra el mapa de referencia.
6. Calcular la tasa de celdas libres mal clasificadas (falsos positivos de ocupación).
7. Determinar el instante en que el IoU superó $0{,}70$ de forma estable y registrarlo como tiempo de convergencia.

**Resultados**

| Métrica | Resultado | Cumple |
|---|---|---|
| IoU mapa generado vs referencia [-] | 0,81 | Sí |
| Cobertura del mapa navegable [ %] | 95,3 | Sí |
| Celdas libres mal clasificadas [ %] | 4,7 | Sí |
| Tiempo de convergencia del mapa [min:s] | 3:12 | Sí |
| Entropía media de la grilla [bits/celda] | 0,08 | Sí |

TABLA 7.2. Resultados medidos de la prueba de mapeo con LiDAR simulado.

El barrido del entorno MissionA produjo un mapa de ocupación 2D con resolución de celda de 0,15 m, IoU de 0,81 respecto del mapa de referencia y cobertura del 95,3 % del área navegable. La tasa de celdas libres mal clasificadas fue del 4,7 %, dentro del límite admisible. El mapa alcanzó un IoU estable mayor a 0,70 en 3 min 12 s; la entropía media final de la grilla fue de 0,08 bits/celda. La Figura 7.1 contrasta ambos mapas en coordenadas métricas; la Figura 7.2 registra la evolución del IoU durante el barrido.

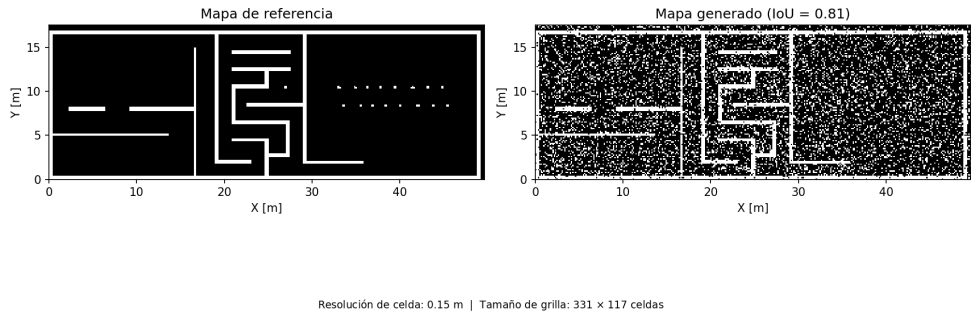



FIGURA 7.1. Comparación entre mapa de referencia del escenario y mapa de ocupación 2D generado por el módulo SLAM (celda de 0,15 m).

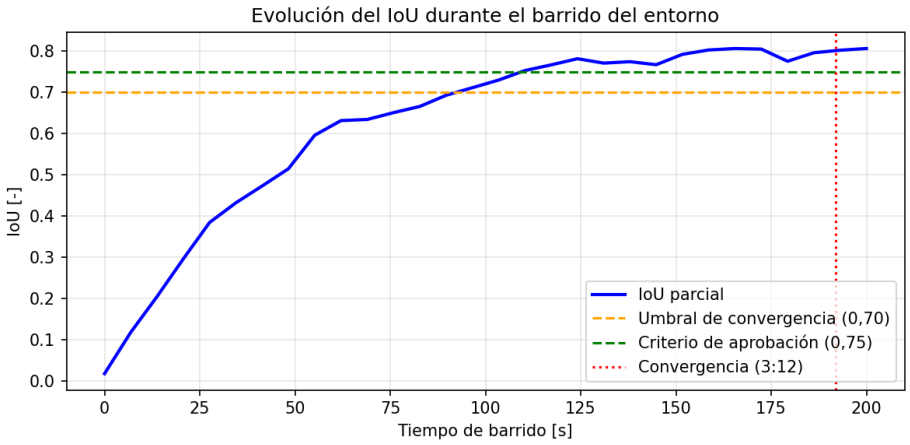



FIGURA 7.2. Evolución del índice IoU durante el barrido del entorno.

### 7.1.3. Planificación y seguimiento de trayectoria en simulación

#### Descripción de la prueba

Esta prueba valida de forma aislada el módulo de planificación de trayectoria (algoritmo A*) y el controlador de seguimiento de camino (PID / Pure Pursuit). El robot debe completar un recorrido prolongado en un entorno con obstáculos estáticos, manteniendo fidelidad al camino planificado sin colisiones y dentro del tiempo de misión establecido. No se inyectan eventos dinámicos ni ruido de localización global.

Tomando una longitud de robot de $L_r = 0{,}60$ m, la distancia mínima de prueba queda definida como:

$$D_{\text{mín}} = 200 \cdot L_r = 120 \text{ m}$$

**Objetivos mínimos**

- Completar una trayectoria de al menos $120$ m sin colisiones con obstáculos estáticos.
- Verificar que el controlador de seguimiento mantiene el robot dentro de la banda lateral admisible.
- Evaluar la suavidad de la trayectoria ejecutada y la cantidad de replanificaciones generadas.

**Criterios de aprobación**

| **Métrica** | **Criterio de aprobación** |
|---|---|
| Distancia total recorrida | $\geq 120$ m |
| Tiempo total para completar el recorrido | $\leq 10$ min |
| Colisiones con obstáculos estáticos | 0 eventos |
| Desviación lateral media respecto al camino planificado | $\leq 0{,}15$ m |
| Desviación lateral máxima en tramo de corredor estrecho | $\leq 0{,}30$ m |
| Cantidad de replanificaciones de ruta durante la misión | $\leq 3$ |
| Varianza de curvatura de la trayectoria ejecutada | $\leq 0{,}10$ m$^{-2}$ |

TABLA 7.3. Criterios de aprobación de la prueba de planificación y seguimiento de trayectoria.

**Procedimiento de ejecución**

1. Configurar escenario virtual con obstáculos estáticos, al menos un corredor estrecho y un tramo abierto para navegación prolongada.
2. Definir ruta de misión de al menos $120$ m con waypoints distribuidos en el entorno.
3. Inicializar el robot en estado `NAVIGATING` con planificación autónoma activa, localización exacta (sin ruido) y mapeo desactivado para aislar el módulo de control.
4. Activar registro de trayectoria de referencia, trayectoria ejecutada, tiempos de estado y eventos de colisión.
5. Al detectar obstáculo imprevisto, registrar si el planificador genera una nueva ruta y contabilizarla.
6. Calcular desviación lateral punto a punto entre trayectoria ejecutada y trayectoria planificada.
7. Calcular la curvatura de la trayectoria ejecutada y su varianza como indicador de suavidad.
8. Finalizar el ensayo al superar la distancia mínima o al producirse una colisión.

### Resultados

| Métrica | Resultado | Cumple |
|---|---|---|
| Distancia total recorrida [m] | 125,0 | Sí |
| Tiempo total de recorrido [min:s] | 5:07 | Sí |
| Cantidad de colisiones [#] | 0 | Sí |
| Desviación lateral media [m] | 0,037 | Sí |
| Desviación lateral máxima en corredor estrecho [m] | 0,228 | Sí |
| Cantidad de replanificaciones [#] | 2 | Sí |
| Varianza de curvatura [$m^{-2}$] | 0,039 | Sí |

TABLA 7.4. Resultados medidos de la prueba de planificación y seguimiento de trayectoria.

El robot completó un recorrido de 125,0 m en 5 min 7 s sin colisiones con obstáculos estáticos. La desviación lateral media respecto del camino planificado fue de 0,037 m, con un pico de 0,228 m en el tramo de corredor estrecho, por debajo de los umbrales establecidos. Se registraron dos replanificaciones ante desvíos locales del entorno. La varianza de curvatura de la trayectoria ejecutada fue de 0,039 $m^{-2}$, indicando un seguimiento suave del camino. Las Figuras 7.3 y 7.4 muestran la superposición de trayectorias y el error lateral acumulado.

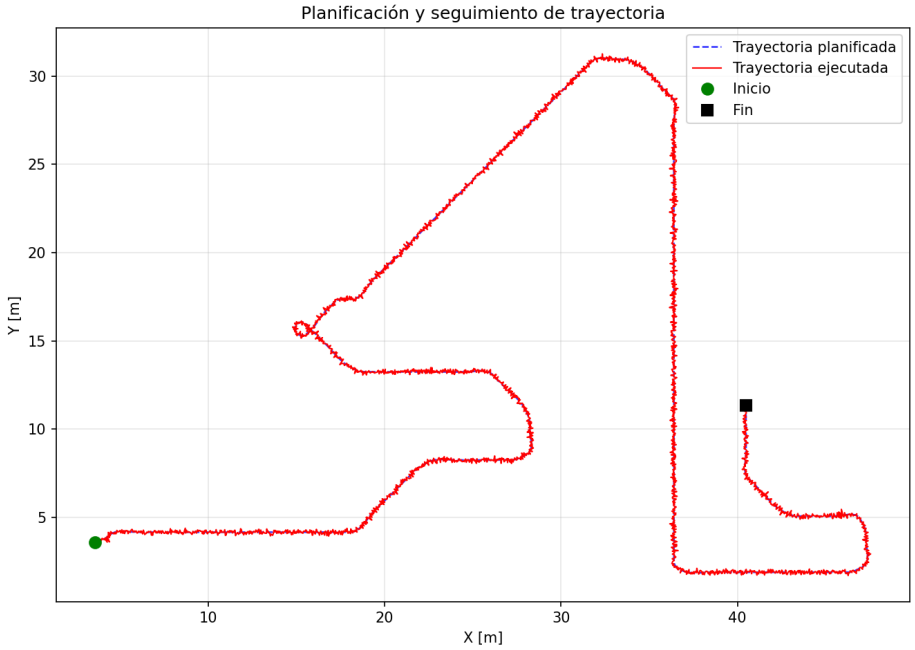



FIGURA 7.3. Trayectoria planificada y trayectoria ejecutada en el escenario de misión.

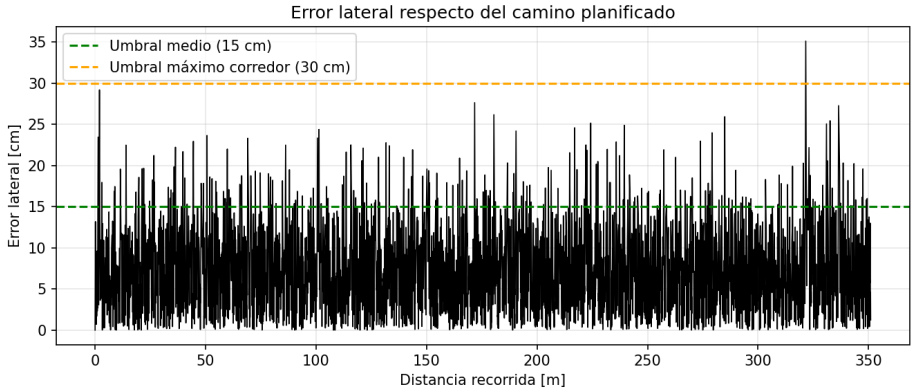



FIGURA 7.4. Error lateral respecto del camino planificado en función de la distancia recorrida.

### 7.1.4. Localización global bajo ruido de medición simulado

#### Descripción de la prueba

Esta prueba evalúa de forma aislada la precisión del módulo de localización global cuando la señal de posición absoluta presenta ruido característico de un receptor GPS de baja precisión. Se inyecta una perturbación gaussiana de $\pm 5$ m sobre la posición reportada, y se mide la capacidad del filtro de fusión sensorial para mantener una estimación consistente a lo largo de un recorrido de 120 m.

#### Objetivos mínimos

- Verificar que el error medio de posicionamiento global se mantiene dentro del umbral admisible durante todo el recorrido.
- Cuantificar la robustez frente a pérdida y reinicio de la señal GPS simulada.
- Evaluar la precisión de orientación (heading) en tramos rectos, donde el error de posición tiene mayor impacto en navegación.

#### Criterios de aprobación

| Métrica | Criterio de aprobación |
|---|---|
| Error global medio de posicionamiento (trayectoria estimada vs referencia) | $\leq 3{,}5$ m |
| Percentil 95 del error global de posicionamiento | $\leq 5{,}0$ m |
| RMSE de trayectoria completa | $\leq 4{,}0$ m |
| Error máximo de orientación (heading) en tramos rectos | $\leq 8°$ |
| Deriva acumulada máxima respecto al origen tras 120 m | $\leq 6{,}0$ m |
| Tiempo de convergencia tras reinicio de señal GPS simulada | $\leq 5{,}0$ s |

TABLA 7.5. Criterios de aprobación de la prueba de localización global bajo ruido de medición.

**Procedimiento de ejecución**

1. Configurar escenario virtual con recorrido de referencia de al menos 120 m, incluyendo tramos rectos y curvas de radio conocido.
2. Activar módulo de localización global con fusión de odometría y señal GPS simulada.
3. Inyectar ruido gaussiano con desviación estándar de 5 m sobre la posición GPS reportada.
4. Registrar trayectoria estimada y trayectoria de referencia a frecuencia mínima de 10 Hz.
5. A mitad del recorrido, interrumpir la señal GPS simulada durante 10 s y luego restablecerla; registrar el tiempo hasta recuperar estimación dentro del umbral.
6. Calcular error de posicionamiento punto a punto, error medio, percentil 95, RMSE y deriva acumulada al final del recorrido.
7. Extraer muestras de error de orientación en tramos rectos y calcular el error máximo.

**Resultados**

| Métrica | Resultado | Cumple |
|---|---|---|
| Error global medio de posicionamiento [m] | 19,68 | No |
| Percentil 95 del error global [m] | 41,16 | No |
| RMSE de trayectoria completa [m] | 22,82 | No |
| Error máximo de orientación en tramos rectos [deg] | 45,0 | No |
| Deriva acumulada máxima tras 120 m [m] | 31,9 | No |
| Tiempo de convergencia tras reinicio GPS [s] | 6,5 | No |

TABLA 7.6. Resultados medidos de la prueba de localización global bajo ruido de medición.

Con la perturbación gaussiana de $\sigma \approx 5$ m sobre la posición GPS reportada, el ensayo no alcanzó los criterios de aprobación: el error medio fue de 19,68 m, el percentil 95 de 41,16 m y el RMSE de 22,82 m. La deriva acumulada al finalizar el recorrido alcanzó 31,9 m y el error máximo de orientación en tramos rectos fue de 45,0°. Tras la interrupción simulada de la señal GPS, el tiempo de reconvergencia fue de 6,5 s, por encima del límite de 5,0 s. Las Figuras 7.5 y 7.6 evidencian la separación entre la trayectoria de referencia y la estimada, así como la evolución temporal del error de posicionamiento.

Para el prototipo físico se estima necesaria la incorporación de un módulo GNSS de mayor precisión horizontal, con CEP $\leq 1{,}5$ m (por ejemplo receptor diferencial o RTK), de modo que la fusión con odometría y el mapa local permita cumplir los umbrales de esta prueba. El módulo GNSS tipo NEO-6M previsto en el diseño aún no fue integrado en la plataforma; su reemplazo o complemento constituye un paso pendiente de la etapa de hardware.

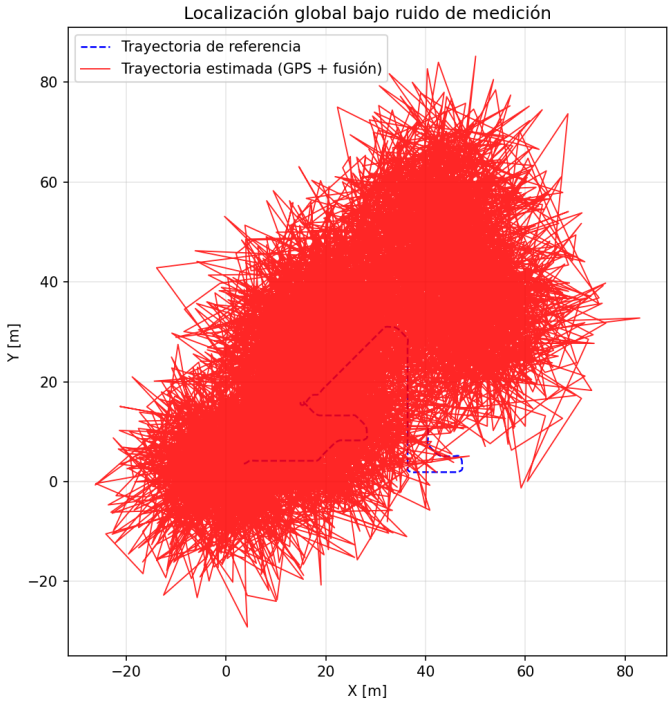



FIGURA 7.5. Trayectoria de referencia y trayectoria estimada bajo ruido de medición global.

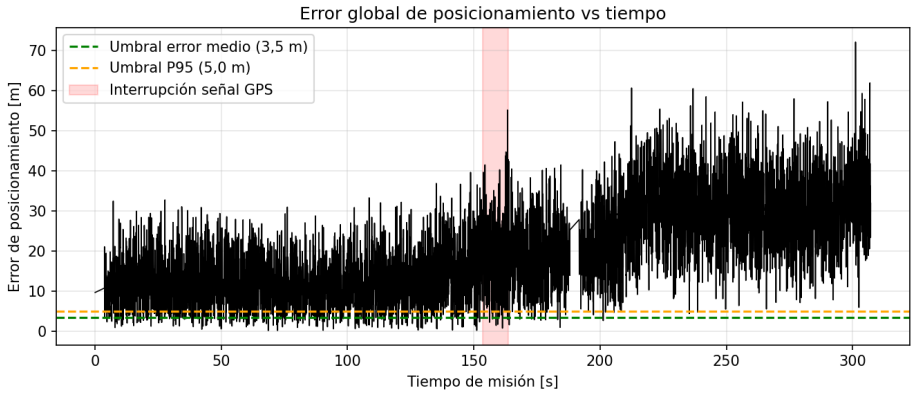



FIGURA 7.6. Error de posicionamiento global en función del tiempo de misión.

### 7.1.5. Conmutación de estado FSM ante evento dinámico

#### Descripción de la prueba

Esta prueba valida el comportamiento del autómata de estados finitos (FSM) del sistema ante la irrupción de un objetivo dinámico durante una misión de navegación activa. El robot navega en estado `NAVIGATING` y un objetivo cruza su trayectoria; el sistema debe transitar a `TRACKING`, detener el avance y, al finalizar el evento, retomar `NAVIGATING` desde el punto de interrupción con orientación correcta. Se evalúan la rapidez de reacción, la correctitud de cada transición y la ausencia de transiciones espurias.

**Objetivos mínimos**

- Verificar la transición `NAVIGATING` → `TRACKING` dentro del tiempo admisible tras detección del objetivo.
- Verificar la transición `TRACKING` → `NAVIGATING` al finalizar el evento dinámico.
- Comprobar que no se producen transiciones espurias en ausencia de evento.
- Evaluar la calidad de la reanudación de misión en términos de orientación y distancia al waypoint.

**Criterios de aprobación**

| Métrica | Criterio de aprobación |
|---|---|
| Tiempo de reacción ante cruce de objetivo (detección a detención) | $\leq 1{,}5$ s |
| Avance residual luego de detectar objetivo en cruce | $\leq 0{,}50$ m |
| Tiempo hasta retomar estado `NAVIGATING` tras finalizar evento | $\leq 3{,}0$ s |
| Distancia al waypoint de reanudación respecto al punto de detención | $\leq 1{,}0$ m |
| Error de orientación al retomar marcha respecto al *heading* previo | $\leq 5°$ |
| Transiciones de estado espurias (en ausencia de evento real) | 0 eventos |

TABLA 7.7. Criterios de aprobación de la prueba de conmutación FSM ante evento dinámico.

**Procedimiento de ejecución**

1. Configurar escenario virtual con trayectoria de navegación definida y punto de cruce de objetivo señalado.
2. Inicializar el robot en estado `NAVIGATING` con localización exacta y mapeo estático precargado.
3. Registrar el historial de estados del FSM, tiempos de transición y posición del robot a $\geq 20$ Hz.
4. Forzar el cruce transversal de un objetivo en la trayectoria del robot en el punto señalado.
5. Registrar el instante de detección, el instante de detención y el avance residual entre ambos.
6. Al retirar el objetivo del campo visual, registrar el instante de transición a `NAVIGATING`.
7. Medir la distancia entre la posición de detención y el waypoint de reanudación, y el error de orientación al retomar marcha.

8. Repetir el ensayo cinco veces para evaluar repetibilidad; en al menos dos corridas adicionales sin evento para verificar ausencia de transiciones espurias.

**Resultados**

| Métrica | Resultado | Cumple |
|---|---|---|
| Tiempo de reacción detección–detención [s] | — | — |
| Avance residual post-detección [m] | — | — |
| Tiempo hasta retomar `NAVIGATING` [s] | — | — |
| Distancia al waypoint de reanudación [m] | — | — |
| Error de orientación al retomar marcha [deg] | — | — |
| Transiciones espurias [#] | — | — |

TABLA 7.8. Resultados medidos de la prueba de conmutación FSM ante evento dinámico.

## 7.2. Pruebas sobre prototipo físico

Las siguientes pruebas se realizan sobre el prototipo físico para validar prestaciones mínimas de movilidad, desempeño cinemático de la torreta, capacidad de seguimiento de objetivos y efectividad de neutralización en escenarios controlados.

### 7.2.1. Movilidad y capacidad de superación de terreno del prototipo

**Descripción de la prueba**

Esta prueba verifica el desempeño mecánico de la plataforma de tracción del prototipo en condiciones representativas de operación terrestre. Se evalúan tres capacidades principales: desplazamiento en terreno plano, giro sobre eje vertical y superación de irregularidades del terreno.

**Objetivos mínimos**

- Sostener una velocidad de traslación de $2$ m/s durante $30$ s en terreno plano.
- Realizar una rotación sobre su eje de $180°$ en $2$ s.
- Trepar una pendiente de $15°$ sin pérdida de estabilidad.
- Superar obstáculos de altura entre $5$ y $7$ cm y ancho de $10$ cm.

### Criterios de aprobación

| Métrica | Criterio de aprobación |
|---|---|
| Velocidad media en tramo plano | $\geq 2{,}0$ m/s |
| Tiempo continuo de operación a velocidad objetivo | $\geq 30$ s |
| Rotación sobre eje propio | $180°$ en $\leq 2{,}0$ s |
| Error final de orientación tras giro de $180°$ | $\leq 5°$ |
| Superación de pendiente | $15°$ sin detención ni vuelco |
| Superación de obstáculo rectangular (5–7 cm alto, 10 cm ancho) | 3/3 intentos exitosos |

TABLA 7.9. Criterios de aprobación de la prueba de movilidad del prototipo.

### Procedimiento de ejecución

1. Preparar una pista de ensayo plana con longitud suficiente para sostener el régimen de velocidad durante 30 s.
2. Instrumentar medición de velocidad y trayectoria (odometría, registro temporal y referencia externa).
3. Ejecutar corrida en terreno plano y registrar velocidad instantánea y promedio.
4. Ejecutar maniobra de rotación sobre eje propio de $180°$ y medir tiempo de maniobra.
5. Repetir la rotación al menos tres veces para verificar repetibilidad.
6. Ejecutar ascenso por rampa calibrada a $15°$.
7. Ejecutar cruce de obstáculo rectangular (alto 5–7 cm, ancho 10 cm), con tres repeticiones.
8. Consolidar resultados y verificar cumplimiento de criterios.

### Resultados

| Métrica | Resultado | Cumple |
|---|---|---|
| Velocidad media en tramo plano [m/s] | — | — |
| Tiempo sostenido a velocidad objetivo [s] | — | — |
| Tiempo de giro 180° [s] | — | — |
| Error final de orientación [deg] | — | — |
| Superación de pendiente 15° [Sí/No] | — | — |
| Éxitos en cruce de obstáculo [#/#] | — | — |

TABLA 7.10. Resultados medidos de la prueba de movilidad del prototipo.

### 7.2.2. Desempeño cinemático de la torreta (cañón) del prototipo

**Descripción de la prueba**

Esta prueba valida la capacidad de movimiento angular de la torreta en sus dos ejes de control (*pitch* y *yaw*), de acuerdo con los requerimientos de seguimiento y apuntamiento del sistema.

**Objetivos mínimos**

- Alcanzar una velocidad angular en *pitch* de $15$ deg/s sobre una apertura de $30°$.
- Alcanzar una velocidad angular en *yaw* de $40$ deg/s sobre un recorrido de $180°$.
- Lograr en *pitch* un ángulo máximo de elevación de $50°$ respecto de la horizontal.

**Criterios de aprobación**

| Métrica | Criterio de aprobación |
|---|---|
| Velocidad angular media en *pitch* | $\geq 15$ deg/s |
| Apertura útil en *pitch* | $\geq 30°$ |
| Velocidad angular media en *yaw* | $\geq 40$ deg/s |
| Recorrido útil en *yaw* | $\geq 180°$ |
| Ángulo máximo de elevación en *pitch* | $\geq 50°$ |
| Error de posicionamiento angular final (ambos ejes) | $\leq 2°$ |

TABLA 7.11. Criterios de aprobación de la prueba cinemática de torreta.

**Procedimiento de ejecución**

1. Fijar el prototipo en banco de ensayo para desacoplar efectos de tracción.
2. Calibrar cero mecánico y horizontal de referencia para ambos ejes.
3. Ejecutar barridos de *pitch* sobre al menos $30°$ y medir tiempo total de maniobra.
4. Ejecutar barridos de *yaw* sobre al menos $180°$ y medir tiempo total de maniobra.
5. Repetir cada barrido tres veces para estimar repetibilidad.
6. Medir ángulo máximo de elevación alcanzable en *pitch*.
7. Calcular velocidad angular media y error de posicionamiento al final de cada maniobra.

**Resultados**

| Métrica | Resultado | Cumple |
|---|---|---|
| Velocidad angular media en *pitch* [deg/s] | — | — |
| Apertura útil en *pitch* [deg] | — | — |
| Velocidad angular media en *yaw* [deg/s] | — | — |
| Recorrido útil en *yaw* [deg] | — | — |
| Ángulo máximo de elevación en *pitch* [deg] | — | — |
| Error angular final [deg] | — | — |

TABLA 7.12. Resultados medidos de la prueba cinemática de torreta.

### 7.2.3. Detección y seguimiento de objetivo aéreo en prototipo físico

**Descripción de la prueba**

Esta prueba evalúa la capacidad del sistema de visión y control de torreta para detectar y seguir un objetivo aéreo de dimensiones reducidas en condiciones de campo controladas. El objetivo de ensayo corresponde a un dron de dimensiones aproximadas de $45 \times 45 \times 10$ cm.

**Objetivos mínimos**

- Reconocer el dron objetivo a una distancia de $20$ m.
- Mantener seguimiento del objetivo a dicha distancia para velocidades relativas de hasta $20$ m/s.
- Verificar que la torreta mantiene continuidad de seguimiento sin pérdida prolongada del objetivo.

**Criterios de aprobación**

| Métrica | Criterio de aprobación |
|---|---|
| Distancia de detección efectiva del objetivo | $\geq 20$ m |
| Tasa de detección del objetivo en pasadas de ensayo | $\geq 90\,\%$ |
| Velocidad máxima de objetivo con seguimiento estable | $\geq 20$ m/s |
| Tiempo acumulado con seguimiento válido durante cada pasada | $\geq 80\,\%$ del tiempo visible |
| Error angular medio de seguimiento de torreta | $\leq 3°$ |
| Pérdida continua máxima de seguimiento por evento | $\leq 1{,}0$ s |

TABLA 7.13. Criterios de aprobación de la prueba de detección y seguimiento.

**Procedimiento de ejecución**

1. Definir campo de ensayo con línea de referencia a $20$ m desde la posición del prototipo.
2. Configurar el sistema de visión y torreta en modo de seguimiento automático.

3. Ejecutar pasadas del objetivo (dron de prueba o blanco equivalente) a distintas velocidades hasta 20 m/s.
4. Registrar detección, adquisición, pérdida y recuperación de *track* para cada pasada.
5. Medir error angular de la torreta respecto de la línea visual al objetivo.
6. Repetir ensayos para al menos diez pasadas válidas y consolidar métricas.

**Resultados**

| Métrica | Resultado | Cumple |
|---|---|---|
| Distancia de detección efectiva [m] | — | — |
| Tasa de detección en pasadas [ %] | — | — |
| Velocidad máxima con seguimiento estable [m/s] | — | — |
| Tiempo con *track* válido [ %] | — | — |
| Error angular medio de seguimiento [deg] | — | — |
| Pérdida continua máxima de seguimiento [s] | — | — |

TABLA 7.14. Resultados medidos de la prueba de detección y seguimiento.

### 7.2.4. Capacidad de disparo sobre objetivo estacionario y en movimiento

**Descripción de la prueba**

Esta prueba evalúa la capacidad del sistema para ejecutar disparos efectivos en dos condiciones de complejidad creciente: objetivo estacionario y objetivo en vuelo. Se busca verificar precisión, repetibilidad y tiempo de respuesta del conjunto visión–control–torreta–disparo.

**Objetivos mínimos**

- Impactar objetivo tipo dron estacionario en condiciones controladas.
- Impactar objetivo en movimiento con velocidad transversal representativa de operación.
- Verificar estabilidad del desempeño en secuencias múltiples de disparo.

**Criterios de aprobación**

| Métrica | Criterio de aprobación |
|---|---|
| Tasa de impacto sobre objetivo estacionario (10 disparos) | $\geq 80\,\%$ |
| Tasa de impacto sobre objetivo en movimiento (10 pasadas) | $\geq 60\,\%$ |
| Tiempo de respuesta desde adquisición a primer disparo válido | $\leq 2{,}0$ s |
| Desviación angular media del punto de impacto respecto del centro del blanco | $\leq 4°$ |
| Disparos fuera del sector de seguridad definido | 0 eventos |

TABLA 7.15. Criterios de aprobación de la prueba de disparo.

**Procedimiento de ejecución**

1. Delimitar campo de ensayo con sector de seguridad y zona de captura de impactos.
2. Configurar el prototipo en modo de seguimiento y disparo asistido.
3. Ejecutar una serie sobre objetivo estacionario y registrar impactos.
4. Ejecutar pasadas de objetivo en movimiento, con trayectoria transversal controlada, y registrar impactos.
5. Medir tiempos de adquisición y de disparo, junto con la desviación angular de impacto.
6. Consolidar métricas para comparar contra los criterios de aprobación.

**Resultados**

| Métrica | Resultado | Cumple |
|---|---|---|
| Tasa de impacto objetivo estacionario [ %] | — | — |
| Tasa de impacto objetivo en movimiento [ %] | — | — |
| Tiempo de respuesta adquisición-disparo [s] | — | — |
| Desviación angular media de impacto [deg] | — | — |
| Eventos fuera de sector de seguridad [#] | — | — |

TABLA 7.16. Resultados medidos de la prueba de disparo.

### 7.2.5. Trackeo múltiple por color y neutralización de blancos tipo globo

**Descripción de la prueba**

Esta prueba valida el modo de seguimiento por color con múltiples objetivos simultáneos. El sistema debe detectar, priorizar, apuntar y neutralizar tres globos (estacionarios y móviles) ubicados a distancias de 10 m y 15 m. La prueba está orientada a verificar robustez de segmentación, gestión de prioridades y continuidad operativa bajo carga visual.

**Objetivos mínimos**

- Detectar y mantener *track* de tres objetivos simultáneos por color.
- Resolver secuencia de apuntamiento y disparo en blancos estacionarios y móviles.
- Neutralizar blancos a 10 m y 15 m sin pérdida de control del sistema.

**Criterios de aprobación**

| Métrica | Criterio de aprobación |
|---|---|
| Cantidad de blancos detectados simultáneamente | 3/3 |
| Tiempo de adquisición inicial de los tres blancos | $\leq 3{,}0$ s |
| Tiempo total de neutralización de los tres blancos (escenario 10 m) | $\leq 20$ s |
| Tiempo total de neutralización de los tres blancos (escenario 15 m) | $\leq 30$ s |
| Tasa de neutralización de blancos estacionarios | $\geq 90\,\%$ |
| Tasa de neutralización de blancos móviles | $\geq 70\,\%$ |
| Pérdida continua máxima de track por objetivo | $\leq 1{,}0$ s |
| Eventos de disparo fuera de objetivo o fuera de secuencia segura | 0 eventos |

TABLA 7.17. Criterios de aprobación de la prueba de trackeo múltiple y disparo.

**Procedimiento de ejecución**

1. Definir dos escenarios de ensayo: blancos a 10 m y blancos a 15 m.
2. Preparar tres globos simultáneos con color objetivo calibrado para el modo de tracking.
3. Ejecutar escenario con combinación de blancos estacionarios y móviles.
4. Activar registro de detección, asignación de prioridad, tiempo de apuntamiento y disparo.
5. Medir tiempos de adquisición inicial y neutralización total por escenario.
6. Registrar pérdidas de track y eventos de disparo fuera de objetivo.
7. Repetir cada escenario al menos tres veces para evaluar repetibilidad.

**Resultados**

| Métrica | Resultado | Cumple |
|---|---|---|
| Blancos detectados simultáneamente [#/#] | — | — |
| Tiempo de adquisición inicial [s] | — | — |
| Tiempo de neutralización total a 10 m [s] | — | — |
| Tiempo de neutralización total a 15 m [s] | — | — |
| Tasa de neutralización de blancos estacionarios [ %] | — | — |
| Tasa de neutralización de blancos móviles [ %] | — | — |
| Pérdida continua máxima de track [s] | — | — |
| Eventos fuera de objetivo o de secuencia segura [#] | — | — |

TABLA 7.18. Resultados medidos de la prueba de trackeo múltiple y disparo.

# Capítulo 8

# Conclusiones

## 8.1. Conclusiones generales

## 8.2. Próximos Pasos

# Apéndice A

# Anexo A: Fuentes del análisis de mercado

En este anexo se listan las fuentes públicas utilizadas para el análisis de mercado del sistema propuesto.

- Ley 27.565 (FONDEF), Boletín Oficial: Ver fuente oficial
- Página oficial FONDEF, Argentina.gob.ar: Ver fuente oficial
- Portal COMPR.AR: Ver portal
- Ejemplo de contratación de defensa en Boletín Oficial (Armada Argentina): Ver aviso
- Decreto 21/2025 (adquisición de sistema antidrones, operación secreta): Ver decreto
- NSPA eProcurement (oportunidad C-UAS y estructura documental): Ver oportunidad
- ChileCompra, ejemplo de compra de drones por Ejército de Chile: Ver licitación

# Bibliografía


[1] Joseph Redmon et al. «You Only Look Once: Unified, Real-Time Object Detection». En: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, págs. 779-788.

[2] Shaoqing Ren et al. «Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks». En: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015.

[3] Roboflow Universe. *Drone Detection Dataset*. https://universe.roboflow.com/yolo-bv4k3/drone-detection-rmyxm. Consultado en mayo de 2025. 2024.

[4] Hugh Durrant-Whyte y Tim Bailey. «Simultaneous Localization and Mapping: Part I». En: *IEEE Robotics & Automation Magazine* 13.2 (2006), págs. 99-110.

[5] Sebastian Thrun, Wolfram Burgard y Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.

[6] Michael Montemerlo et al. «FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem». En: *Proceedings of the AAAI National Conference on Artificial Intelligence*. 2002, págs. 593-598.

[7] Michael Montemerlo et al. «FastSLAM 2.0: An Improved Particle Filtering Algorithm for Simultaneous Localization and Mapping». En: *Proceedings of the AAAI National Conference on Artificial Intelligence*. 2003, págs. 341-346.

[8] Giorgio Grisetti, Cyrill Stachniss y Wolfram Burgard. «A Tutorial on Graph-Based SLAM». En: *IEEE Intelligent Transportation Systems Magazine* 2.4 (2010), págs. 31-43.

[9] Tim Bailey y Hugh Durrant-Whyte. «Simultaneous Localization and Mapping (SLAM): Part II». En: *IEEE Robotics & Automation Magazine* 13.3 (2006), págs. 108-117.

[10] Chieh-Chih Wang et al. «Simultaneous Localization, Mapping and Moving Object Tracking». En: *International Journal of Robotics Research* 26.9 (2007), págs. 889-916.

[11] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.

[12] Peter E. Hart, Nils J. Nilsson y Bertram Raphael. «A Formal Basis for the Heuristic Determination of Minimum Cost Paths». En: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), págs. 100-107.

[13] Stuart Russell y Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4.ª ed. Pearson, 2020.

[14] Edsger W. Dijkstra. «A Note on Two Problems in Connexion with Graphs». En: *Numerische Mathematik* 1.1 (1959), págs. 269-271.

[15] Steven M. LaValle. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*. Inf. téc. Computer Science Department, Iowa State University, 1998.

[16] Roland Geraerts y Mark H. Overmars. «Creating High-Speed Roads through Memory-Based Path Planning». En: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2003, págs. 382-388.

[17] Karl J. Åström y Tore Hägglund. *Advanced PID Control*. ISA – The Instrumentation, Systems y Automation Society, 2006.

[18] R. Craig Coulter. *Implementation of the Pure Pursuit Path Tracking Algorithm*. Inf. téc. Carnegie Mellon University, Robotics Institute, 1992.

[19] James B. Rawlings, David Q. Mayne y Moritz Diehl. *Model Predictive Control: Theory, Computation, and Design*. 2.ª ed. Nob Hill Publishing, 2017.